



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

DEPARTMENT OF INFORMATION SYSTEMS

REPREZENTACE SÍŤOVÝCH TOKŮ S VYUŽITÍM NEURONOVÝCH SÍTÍ

REPRESENTATION OF NETWORK FLOWS USING NEURAL NETWORKS

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

LUKASZ PYCZ

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. DANIEL POLIAKOV

BRNO 2024

Zadání bakalářské práce



156745

Ústav: Ústav informačních systémů (UIFS)
Student: **Pycz Lukasz**
Program: Informační technologie
Název: **Reprezentace síťových toků s využitím neuronových sítí**
Kategorie: Umělá inteligence
Akademický rok: 2023/24

Zadání:

1. Seznamte se s možnostmi statistické analýzy na úrovni síťových toků. Prozkoumejte běžné charakteristiky využívané v rámci analýzy. Nastudujte si metody self-supervised learningu pro extrakci reprezentace dat.
2. Na základě získaných znalostí navrhnete model pro extrakci reprezentací síťových toků.
3. Model experimentálně implementujte a natrénujte s využitím knihovny PyTorch.
4. Za použití datasetu síťových služeb získané reprezentace vyhodnotte.
5. Diskutujte dosažené výsledky a možné budoucí rozšíření.

Literatura:

- Hofstede, R.; Čeleda, P.; et al. Flow Monitoring Explained: From Packet Capture to Data Analysis With NetFlow and IPFIX. IEEE Communications Surveys & Tutorials, volume 16, no. 4, 2014: pp. 2037–2064, doi:10.1109/COMST.2014.2321898.
- Yamansavascular, B., Guvensan, M. A., Yavuz, A. G., & Karşlıgil, M. E. (2017, January). Application identification via network traffic classification. In 2017 International Conference on Computing, Networking and Communications (ICNC) (pp. 843-848). IEEE.
- Balestrieri, R.; et al. A cookbook of self-supervised learning. *arXiv preprint arXiv:2304.12210* (2023). <https://doi.org/10.48550/arXiv.2304.12210>

Při obhajobě semestrální části projektu je požadováno:

Splnění bodů 1 až 2 zadání. Rozpracování bodu 3.

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Poliakov Daniel, Ing.**
Vedoucí ústavu: Kolář Dušan, doc. Dr. Ing.
Datum zadání: 1.11.2023
Termín pro odevzdání: 9.5.2024
Datum schválení: 26.10.2023

Abstrakt

Táto bakalárska práca skúma aplikáciu metód samo-riadeného učenia (SSL) ako je maskovanie dát, miešanie poradia dát a využitie kontrastného učenia, na extrakciu zmysluplných reprezentácií z údajov o sieťovom toku, konkrétne s využitím datasetu CESNET TLS22 z CESNET DataZoo. Hlavným cieľom je vyvinúť robustné modely, ktoré zlepšia pochopenie a analýzu sieťových tokov prostredníctvom efektívneho učenia reprezentácie bez závislosti na označených údajoch. Výskum využíva výpočtový rámec PyTorch na návrh, tréning a hodnotenie výkonnosti modelov.

Abstract

This thesis explores the application of self-supervised learning (SSL) methods such as data masking, data order shuffling, and contrastive learning, to extract meaningful representations from network flow data, specifically using the CESNET TLS22 dataset from CESNET DataZoo. The main goal is to develop a robust model that improves the understanding and analysis of network flows through effective representation learning without relying on labeled data. The research utilizes the PyTorch computational framework for designing, training, and evaluating the performance of the model.

Kľúčové slová

Samo-riadené učenie, Analýza sieťového toku, Reprezentácia údajov, PyTorch, CESNET TLS22 Dataset, Neučené učenie, Kontrastívne učenie, Maskovanie údajov, Extrakcia reprezentácií dát, Sieťová bezpečnosť, Neurónové siete

Keywords

Self-Supervised Learning, Network Flow Analysis, Data Representation, PyTorch, CESNET TLS22 Dataset, Unsupervised Learning, Contrastive Learning, Data Masking, Feature Extraction, Network Security, Neural Networks

Citácia

PYCZ, Lukasz. *Reprezentace síťových toků s využitím neuronových sítí*. Brno, 2024. . Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce Ing. Daniel Poliakov

Reprezentace síťových toků s využitím neuronových sítí

Prehlásenie

Prehlasujem, že som túto bakalársku prácu vypracoval samostatne pod vedením pána Ing. Daniela Poliakova. Uviedol som všetky literárne pramene, publikácie a ďalšie zdroje, z ktorých som čerpal.

.....
Lukasz Pycz
9. mája 2024

Podakovanie

Rád by som poďakoval svojmu vedúcemu práce Ing. Danielovi Poliakovovi za pomoc a ochotu pri vedení mojej bakalárskej práce. Tiež by som rád poďakoval mojej rodine a blízkym za ich podporu.

Obsah

1	Úvod	5
2	Strojové učenie	6
2.1	Princip strojového učenia	6
2.2	Typy strojového učenia	6
2.3	Učenie s učiteľom	7
2.3.1	Lineárna regresia	7
2.3.2	Logistická regresia	8
2.3.3	Rozhodovacie stromy	9
2.3.4	Support vector machines (SVM)	9
2.3.5	Neurónové siete	10
2.3.6	Náhodne lesy	10
2.4	Self-supervised učenie	11
2.4.1	Kontrastné učenie	11
2.4.2	Prediktívne učenie	12
2.4.3	Zhlukovanie	12
2.4.4	Generatívne modely	12
2.4.5	Hybridné modely	12
2.4.6	Predpovedanie transformácie	13
2.4.7	Úlohy Pretextu (Pretext Tasks)	13
2.4.8	Downstream Úlohy (Downstream Tasks)	14
2.5	Strojové učenie, Deep learning a Neurónove siete	14
2.6	Neurónove Siete	15
2.7	Aktivačné funkcie	17
2.7.1	Sigmoidná Funkcia	17
2.7.2	Hyperbolická Tangentná Funkcia (Tanh)	18
2.7.3	ReLU (Rectified Linear Unit)	18
2.7.4	Leaky ReLU	18
2.7.5	Softmax	18
2.8	Optimizátory	19
2.8.1	Gradientový zostup	19
2.8.2	Stochastický gradientový zostup (SGD)	20
2.8.3	Adam (Adaptive Moment Estimation)	20
2.8.4	RMSprop (Root Mean Square Propagation)	20
2.9	Funkcie straty	20
2.9.1	Mean Squared Error (MSE)	20
2.9.2	Cross-Entropy Loss	21

2.9.3	Hinge Loss	21
2.9.4	Kullback-Leibler Divergence	21
2.10	Hodnotiace metriky	22
2.10.1	Accuracy	22
2.10.2	Precision) a Recall	22
2.10.3	F1-Score	22
2.10.4	ROC krivka (Receiver Operating Characteristic curve) a ROC AUC (Area Under the Curve)	22
3	Návrh Implementácie	23
3.1	Výber Technológií	24
3.1.1	PyTorch	24
3.1.2	CESNET DataZoo	24
3.1.3	Jupyter Notebook a Vzdialený Počítač	25
3.1.4	Dodatočné Knižnice	25
3.2	Vstupné Dáta	25
3.2.1	CESNET TLS22	25
3.2.2	Práca s dátami	26
3.3	Architektúra Modelov	27
3.3.1	Prvý Model - Maskovanie vs Premiešavanie PPI	27
3.3.2	Druhý Model - Maskovanie vs Kontrastné Učenie	28
3.3.3	Tretí Model - Klasifikačná Úloha vs Kontrastné Učenie s Označeniami	28
3.3.4	Spoločná Architektúra a Evaluácia Modelov	28
4	Implementácia	29
4.1	Pretextové úlohy	29
4.1.1	Pretextová Úloha: Zamiešanie	29
4.1.2	Pretextová Úloha: Maskovanie	29
4.1.3	Kontrastné učenie	29
4.2	Funkcia na extrakciu dát	30
4.3	Model 1 - Kombinovaný SSL Model	30
4.3.1	Architektúra Modelu	30
4.3.2	Tréningový proces	31
4.4	Model 2 - Kombinácia maskovania a kontrastného učenia	32
4.4.1	Architektúra modelu	32
4.4.2	Tréningový proces	32
4.5	Model 3 - Kombinácia klasifikácie a kontrastného učenia	32
4.5.1	Architektúra modelu	32
4.6	Tréningový proces	33
4.6.1	Early Stopping	33
4.6.2	Kombinácia tréningového a validačného datasetu	33
4.6.3	Learning Rate Scheduler	34
4.7	Extrakcia reprezentácii	34
4.8	Downstream úloha	35
4.8.1	Tréningový proces	35
4.8.2	Testovací proces	35
4.8.3	Vyhodnocovanie výkonu	35

5	Výsledky	36
5.1	Experiment.1 - Maskovacia strata vs premiešavacia strata	36
5.1.1	Experimentálny dizajn	36
5.1.2	Hodnotenie výsledkov	36
5.2	Experiment.2 - Maskovacia strata vs kontrastná strata	38
5.2.1	Experimentálny dizajn	38
5.2.2	Hodnotenie výsledkov	38
5.3	Experiment.3 - Klasifikačná strata vs kontrastná strata	40
5.3.1	Experimentálny dizajn	40
5.3.2	Hodnotenie výsledkov	40
6	Záver	43
	Literatúra	45
A	Obsah elektronickej prílohy	48

Zoznam obrázkov

2.1	Príklad jednoduchej lineárnej regresie, s jednou nezávislou premennou. [32]	8
2.2	Graf logistickej regresnej krivky prispôbenej dátam (x_m, y_m) . Krivka ukazuje pravdepodobnosť úspešného zloženia skúšky v závislosti od počtu hodín štúdia.[33]	8
2.3	Príklad rozhodovacieho stromu na rozhodovanie o surfovaní. [9]	9
2.4	Maximálna hraničná hyperrovina a okraje pre SVM trénované s vzorkami z dvoch tried. Vzorky na okraji sa nazývajú support vektory .[34]	9
2.5	Príklad architektúry neuronovej siete [7]	10
2.6	Príklad architektúry náhodných stromov [10]	11
2.7	Princíp kontrastného učenia [30]	11
2.8	Pretextové učenie [17]	13
2.9	Downstream Task [17]	14
2.10	Porovnanie ľudského neurónu a matematického modelu neurónu [28]	15
2.11	Princíp aktivačných funkcií v neuronových sieťach [26]	17
2.12	Príklady aktivačných funkcií [21]	19
3.1	Návrh architektury riešenia	24
4.1	Príklad overfittingu pri trénovaní modelov [13]	34
5.1	Matica zámien - Presnosť správnych predikcií aplikáci pre najvýkonnejší maskovacii a premiešavací model	37
5.2	Graf vplyvu pomeru maskovaciej a premiešavaciej straty na výkon modelu	37
5.3	Matica zámien - Presnosť správnych predikcií aplikáci pre najvýkonnejší maskovacii a kontrastný model	39
5.4	Graf vplyvu pomeru maskovaciej a kontrastnej straty na výkon modelu	39
5.5	Matica zámien - Presnosť správnych predikcií aplikáci pre najvýkonnejší klasifikačný a kontrastný model	41
5.6	Graf vplyvu pomeru klasifikačnej a kontrastnej straty na výkon modelu	41

Kapitola 1

Úvod

[2] Príchod strojového učenia označil novú éru v rôznych oblastiach vedy a technológie, pričom jeho dôsledky rezonujú aj v doméne bezpečnosti sietí. Strojové učenie skúma schopnosť strojov učiť sa a prispôbovať bez explicitného programovania pre špecifické úlohy. Táto práca sa zaoberá zložitostami strojového učenia s cieľom odhaliť vzorce a poznatky v dátovom súbore CESNET-TLS22 so zameraním na posilnenie protokolov bezpečnosti sietí.

Strojové učenie, podmnožina umelej inteligencie, je založené na predpoklade, že systémy môžu učiť z dát, identifikovať vzorce a robiť rozhodnutia s minimálnym ľudským zásahom. Cesta do strojového učenia je zakorenená v hľadaní spôsobu, ako napodobniť ľudské učebné procesy v strojoch, čím sa vytvárajú paralely medzi výpočtovými modelmi a biologickými mechanizmami učenia. Toto úsilie sľubuje nielen zvýšenie existujúcich schopností strojov, ale tiež ponúka pohľad na povahu ľudského učenia.

Táto práca je osobitne zameraná na využitie úloh samoučeného učenia, ako je maskovanie, premiešavanie poradia paketových informácií a kontrastné učenie, na extrakciu reprezentácií z dát netflow. Dátový súbor CESNET-TLS22, bohatý na informácie o sieťových tokoch, poskytuje bezprecedentnú príležitosť na aplikáciu techník strojového učenia pre klasifikáciu sieťových tokov na základe aplikačných značiek. Práca predpokladá, že prostredníctvom SSL môžu stroje využívať neoznačené dáta na zachytenie základných vzorcov v sieťových tokoch, čím umožňujú presnejšiu klasifikáciu aplikácií.

Význam strojového učenia okrem teoretického skúmania, nájde praktické uplatnenie v rôznych oblastiach vrátane autonómnych vozidiel, systémov na rozpoznávanie tvárí a najmä v bezpečnosti sietí. Vzhľadom na vyvoj zložitosti kybernetických hrozieb, stáva sa nevyhnutnou potreba adaptívnych a inteligentných bezpečnostných systémov. Využitím strojového učenia sa táto práca snaží prispieť k vývoju robustných bezpečnostných mechanizmov schopných proaktívne identifikovať a zmierňovať potenciálne hrozby.

Táto práca vedená niekoľkými kľúčovými cieľmi: hodnotiť účinnosť rôznych úloh SSL, po extrakcii reprezentácií tokov, skúmať aplikáciu týchto extrahovaných prvkov pri klasifikácii aplikácií v dátovom súbore CESNET-TLS22 a posúdiť potenciál modelov strojového učenia na prispôbenie sa meniacim sieťovým prostrediam. Cez toto výskum, aspirujeme nielen na posunutie pochopenia strojového učenia v oblasti bezpečnosti sietí, ale tiež na polozenie základov pre budúce inovácie v tomto odbore.

Kapitola 2

Strojové učenie

[8]Strojové učenie, kľúčová podoblasť umelej inteligencie, zahŕňa vývoj algoritmov, ktoré umožňujú počítačom učiť sa z údajov a na ich základe robiť predpovede alebo rozhodnutia. Strojové učenie a rozpoznávanie vzorov sú zásadne spojené s myšlienkou extrahovania užitočných informácií z veľkých dátových súborov, čo je proces nevyhnutný pre pochopenie prirodzených aj umelých systémov.

2.1 Princíp strojového učenia

[29]Strojové učenie funguje prostredníctvom troch hlavných častí učiaceho sa systému algoritmu strojového učenia.

- Rozhodovací proces: Vo všeobecnosti sú algoritmy strojového učenia používané na predikciu alebo klasifikáciu. Na základe určitých vstupných dát, ktoré môžu byť označené alebo neoznačené, algoritmus slúži na produkciu odhadu vzoru v dátach.
- Funkcia chyby: Funkcia chyby hodnotí predikciu modelu. Ak sú známe príklady, funkcia chyby môže urobiť porovnanie na posúdenie presnosti modelu.
- Proces optimalizácie modelu: Ak sa model môže lepšie prispôbiť dátovým bodom v tréningovej sade, potom sa váhy upravujú na zníženie rozdielu medzi známym príkladom a odhadom modelu. Algoritmus bude opakovať tento iteratívny proces "hodnotenia a optimalizácie", aktualizujúc váhy autonómne, až kým nebude dosiahnutý prah presnosti.

Základom strojového učenia je používanie algoritmov na extrahovanie informácií z neupravených údajov a ich reprezentáciu v určitom type modelu.

2.2 Typy strojového učenia

[8][29]Stratégie strojového učenia sú široko kategorizované do troch hlavných typov, pričom každý z nich má jedinečné metodológie a aplikácie:

- Učenie s učiteľom (Supervised Learning): Tento prístup zahŕňa tréning modelu na dátovej sade, ktorá obsahuje páry vstupov a výstupov, pričom výsledky sú už známe. Model sa učí porovnávaním svojho výstupu so správnym výstupom, aby našiel chyby a podľa toho sa upravil.

- Učenie bez učiteľa (Unsupervised Learning): Pri neučenom učení trénovacie dáta zostávajú zo vstupov bez explicitne poskytnutých správnych výstupov. Model sa snaží nájsť vzory a vzťahy v dátach, kategorizuje ich do skupín alebo ich rozkladá podľa spektra, v závislosti od vnútorných štruktúr, ktoré objaví.
- Učenie posilňovaním (Reinforcement Learning): Tento typ učenia využíva mechanizmy podobné tým, ktoré sa nachádzajú v prirodzených učiacich sa systémoch. Modely sa učia robiť postupnosť rozhodnutí, pričom dostávajú spätnú väzbu v podobe odmien alebo trestov. Cieľom je vyvinúť stratégiu, ktorá maximalizuje celkovú odmenu.

2.3 Učenie s učiteľom

[4][29] Učenie s učiteľom zahŕňa trénovanie modelu na mapovanie vstupov na výstupy na základe dvojíc vstup - výstup poskytnutých v trénovacích dátach. Tieto páry sú označené známym „učiteľom“, ktorý pozná správne výstupy pre dané vstupy. Model sa učí aproximáciou funkcie, ktorá mapuje vstup na výstup a minimalizuje chybu medzi svojimi predpoveďami a skutočnými výsledkami. V riadenom učení s učiteľom sa používajú rôzne algoritmy na vytváranie modelov, ktoré predpovedajú výstup spojený s novými vstupnými dátami na základe historických dát. Medzi algoritmy učenia s učiteľom patria:

2.3.1 Lineárna regresia

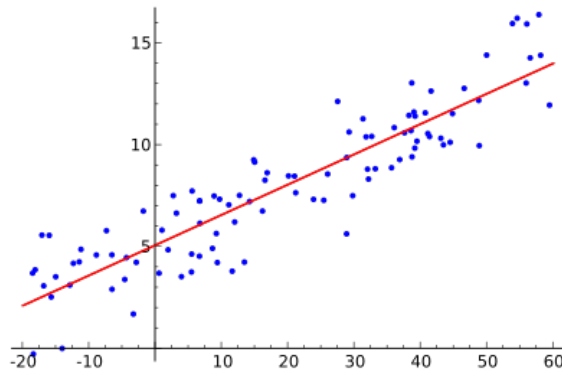
[29] Lineárna regresia predpovedá spojitú výstupnú premennú na základe jednej alebo viacerých vstupných premenných. Predpokladá sa, že vzťah medzi vstupmi a výstupom je lineárny. Model lineárnej regresie možno reprezentovať ako:

$$y = w^T x + b \quad (2.1)$$

kde w predstavuje vektor váh, x vektory vstupných premenných a b posun. Lineárna regresia obvykle používa metódu najmenších štvorcov 2.9.1 na nájdenie najlepšie priliehajúcej priamky. Cieľom je minimalizovať súčet štvorcových rozdielov medzi pozorovanými hodnotami a tými, ktoré predpovedá model:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.2)$$

kde y_i sú skutočné hodnoty a $\hat{y}_i$ sú predpovedané hodnoty. Optimalizácia sa často vykonáva pomocou metódy gradientného zostupu 2.8.1.



Obr. 2.1: Príklad jednoduchkej lineárnej regresie, s jednou nezávislou premennou. [32]

2.3.2 Logistická regresia

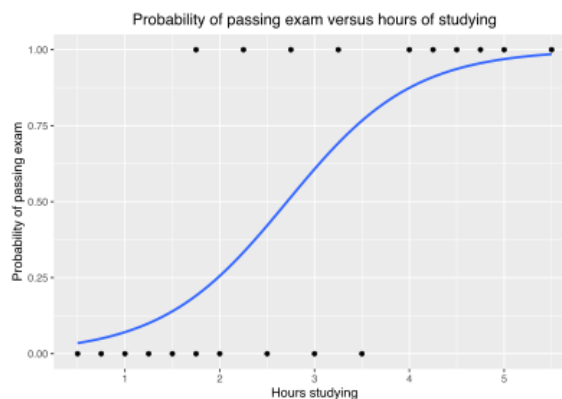
[29] Napriek svojmu názvu sa logistická regresia používa pre úlohy binárnej klasifikácie, nie regresie. Predpovedá pravdepodobnosť, že pozorovanie patrí do jednej z dvoch tried.

$$p(y = 1 \mid x) = \frac{1}{1 + e^{-w^T x}} \quad (2.3)$$

Tu $p(y=1 \mid x)$ je pravdepodobnosť, že klasifikačný štítok je 1 pri daných prvkoch x , a w je vektor váh. Logistická regresia minimalizuje Cross-Entropy stratu 2.9.2, často pomocou metód ako gradientný zostup 2.8.1 :

$$\text{Cross-Entropy} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Bežne sa používa v medicínskych oblastiach (diagnostika chorôb), financiách (hodnotenie úverovej spôsobilosti) a pri detekcii spamu v e-mailoch.



Obr. 2.2: Graf logistickej regresnej krivky prispôbenej dátam (x_m, y_m) . Krivka ukazuje pravdepodobnosť úspešného zloženia skúšky v závislosti od počtu hodín štúdia. [33]

2.3.3 Rozhodovacie stromy

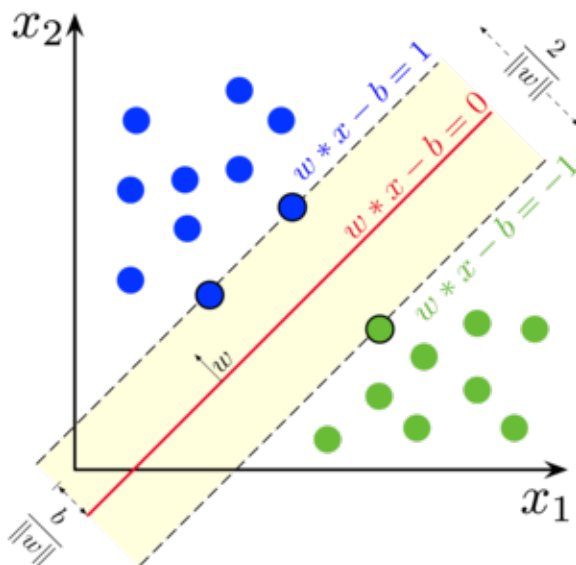
[29] [20] Rozhodovacie stromy používajú stromový model rozhodnutí a ich možných dôsledkov. Ide o sériu pravidiel rozhodnutí typu *if-then-else*. Začína sa od koreňového uzla a údaje sa rozdeľujú na prvok, ktorý na každej úrovni spôsobí najväčší zisk informácií. Strom sa buduje rekurentným rozdeľovaním údajov na podmnožiny, ktoré obsahujú inštancie s podobnými hodnotami (homogénne). Algoritmy *ID3*, *C4.5* alebo *CART* definujú, ako vykonať tieto rozdelenia. Tento algoritmus sa používa v riadení vzťahov s klientmi, diagnostike porúch a v prípadoch kedy je dôležitá transparentnosť rozhodovania.



Obr. 2.3: Príklad rozhodovacieho stromu na rozhodovanie o surfovaní. [9]

2.3.4 Support vector machines (SVM)

[20] SVM sú súbor metód riadeného učenia používaných pre klasifikáciu, regresiu a detekciu odľahlých hodnôt. SVM sa snažia nájsť hyperrovinu v N -rozmernom priestore, ktorá jasne klasifikuje dátové body. Cieľom je nájsť rovinu, ktorá má maximálnu maržu, t.j. maximálnu vzdialenosť medzi dátovými bodmi oboch tried. Maximalizuje maržu okolo rozdeľujúcej hyperroviny. Využíva sa pri úlohach ako detekcia tváre, rozpoznávanie rukopisu a klasifikácia obrázkov.



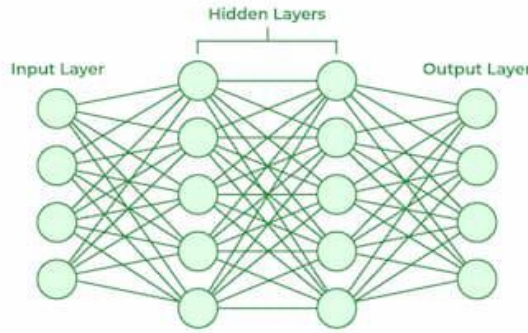
Obr. 2.4: Maximálna hraničná hyperrovina a okraje pre SVM tréňované s vzorkami z dvoch tried. Vzorky na okraji sa nazývajú support vektory. [34]

2.3.5 Neurónové siete

[11] Neurónové siete 2.6 pozostávajú z vrstiev prepojených uzlov alebo neurónov, kde každá vrstva vykonáva rozličné transformácie na vstupných dátach, aby extrahovala kľúčové prvky. Každý neurón v sieti vypočíta vážený súčet svojho vstupu, ktorý je prevedený cez nelineárnu aktivačnú funkciu, typicky ReLU alebo sigmoid 2.7 .

$$a^{(l+1)} = \sigma \left(W^{(l)} a^{(l)} + b^{(l)} \right) \quad (2.4)$$

kde σ je aktivačná funkcia, W váhová matica, a aktivácia z predchádzajúcej vrstvy a b posun. Neurónové siete sú typicky trénované pomocou metódy spätného šírenia, ktorá zahŕňa výpočet gradientu stratovej funkcie vo vzťahu k váham siete pomocou reťazového pravidla a použitie tejto informácie na aktualizáciu váh metódou ako je gradientný zostup 2.8.1. Rozsiahle používané pri rozpoznávaní obrazov a reči, hrách a spracovaní prirodzeného jazyka.



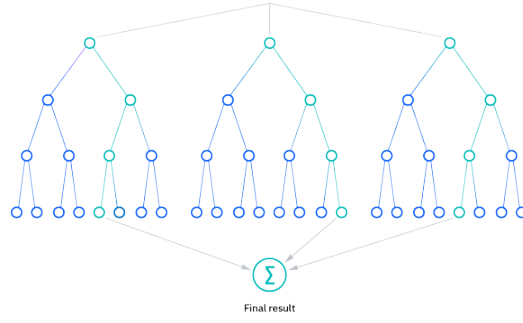
Obr. 2.5: Príklad architektúry neuronovej siete [7]

2.3.6 Náhodne lesy

[12] Náhodný les je metóda učenia ansamble pre klasifikáciu a regresiu, ktorá funguje stavbou viacerých rozhodovacích stromov v čase tréningu.

$$Y = \frac{1}{B} \sum f_b(X) \quad (2.5)$$

kde B je počet stromov, f_b je b -tý rozhodovací strom, a X sú vstupné prvky. Znižuje nadmerné prispôbenie v rozhodovacích stromoch priemerovaním viacerých hlbokých rozhodovacích stromov, trénovaných na rôznych častiach rovnakého tréningového súboru, s cieľom znížiť variáciu. Používa sa na burze na modelovanie očakávaného výnosu z akcie na základe vstupných prvkov, ako sú metriky výkonnosti spoločnosti, historické ceny atď.



Obr. 2.6: Príklad architektúry náhodných stromov [10]

2.4 Self-supervised učenie

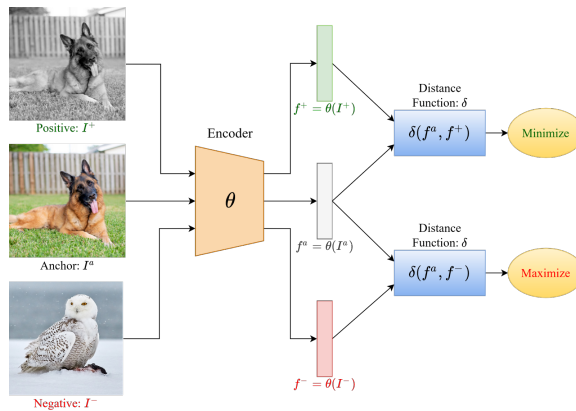
[2][31] Self-supervised učenie je forma učenia bez učiteľa, pri ktorom systém učí sa z dát, ktoré nie sú označené ľuďmi. Používa dataset na vytvorenie pseudo-značiek alebo pretextových úloh, ktoré simulujú scenár učenia s učiteľom. Cieľom SSL je naučiť sa užitočné reprezentácie z veľkého množstva neoznačených dát tým, že predpovedá nepozorovanú alebo skrytú časť vstupných dát z pozorovanej časti. Medzi najpopulárnejšie metódy a techniky self-supervised učenia patria:

2.4.1 Kontrastné učenie

[6] Pri kontrastnom učení sa model trénuje pomocou párov príkladov, ktoré sú buď podobné alebo rozdielne. Cieľom je naučiť sa vektorový priestor, v ktorom sú podobné položky bližšie k sebe a rozdielne položky ďalej od seba. Často používanou strátovou funkciou v tejto metóde je kontrastná strata alebo tripletová strata. Pre pár podobných položiek x_i a x_j môže byť kontrastná strata definovaná ako:

$$L(x_i, x_j) = -\log \frac{\exp(\text{sim}(x_i, x_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(x_i, x_k)/\tau)}$$

Tu $\text{sim}(x_i, x_j)$ je miera podobnosti (často skalárny súčin) medzi vstupmi x_i a x_j , a τ je parameter teploty, ktorý kontroluje oddelovaciu hranicu.



Obr. 2.7: Princíp kontrastného učenia [30]

2.4.2 Prediktívne učenie

[2][18] Prediktívne modely sa učia predpovedaním niektorých aspektov vstupných údajov z iných častí. V kontexte textu by to mohlo zahŕňať predpovedanie ďalšieho slova alebo chýbajúcich slov. Pri obrázkoch by to mohlo zahŕňať predpovedanie ďalšieho snímku vo videu alebo doplnenie zakrytých častí obrázku.

Maskované modelovanie jazyka (BERT):

BERT využíva koncept maskovaného modelovania jazyka, kde sú náhodné slová vo vete nahradené tokenom [MASK], a model sa učí predpovedať pôvodné slovo na základe kontextu poskytnutého ostatnými slovami.

$$P(\text{slovo}|\text{kontext})$$

2.4.3 Zhhlukovanie

[2][18] Pri zhľukovaní self-supervised učení sa model zoskupuje údaje do zhľukov bez použitia štítkov, a tieto priradenia zhľukov sú potom použité ako štítky na tréning modelu v diskriminačnej úlohe.

DeepCluster:

DeepCluster striedavo zhľukuje reprezentácie prvkov údajov a aktualizuje váhy siete predpovedaním priradení do zhľukov.

$$L = \sum_i \|f(x_i) - c_{y_i}\|^2$$

kde $f(x_i)$ je vektor prvkov vstupu x_i , c_{y_i} je centroid zhľuku y_i , do ktorého je x_i priradený.

2.4.4 Generatívne modely

[2][18] Generatívne modely sa učia modelovať distribúciu vstupných údajov a generovať nové údaje, ktoré by mohli pravdepodobne pochádzať z rovnakej distribúcie. Dva populárne typy sú GANy a VAE.

Strata VAE:

$$L = -E_{q(z|x)}[\log p(x|z)] + KL(q(z|x)||p(z))$$

Táto rovnica kombinuje stratu rekonštrukcie (prvý člen) a KL divergenciu medzi naučenou distribúciou latentných premenných $q(z|x)$ a predvolenou distribúciou $p(z)$, čo podporuje kódovanie zmysluplných prvkov v latentnom priestore.

2.4.5 Hybridné modely

[2][18] Hybridné modely integrujú viaceré stratégie self-supervised učenia, aby využili ich príslušné silné stránky. Napríklad model by mohol využívať prediktívne aj kontrastné úlohy, aby zabezpečil robustné učenie prvkov.

2.4.6 Predpovedanie transformácie

[2][18] Táto metóda zahŕňa aplikáciu transformácie na vstupné údaje a tréning modelu na predpovedanie typu transformácie alebo jej parametrov. Tým sa model učí invariantné prvky, ktoré sa nemenia s transformáciou.

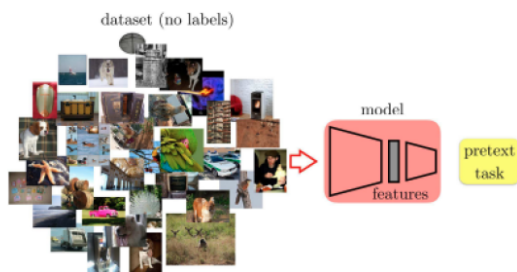
Predpoveď rotácie:

Bežnou úlohou je otočiť obrázok jedným z niekoľkých preddefinovaných uhlov a trénovať model na predpovedanie uhla. Týmto sa model učí rozpoznávať orientáciu, čo je užitočná invariantná vlastnosť vo veľa vizuálnych úlohách.

2.4.7 Úlohy Pretextu (Pretext Tasks)

[2][18] Úlohy pretextu sú umelé úlohy vytvorené špeciálne na účel tréningu strojových učiacich modelov. Kľúčový nápad spočíva v generovaní učebného scenára z neoznačených údajov, kde model môže vyvíjať reprezentácie alebo vlastnosti, ktoré sú užitočné pre skutočné úlohy, ktoré neskôr vykoná. Značky pre tieto úlohy sú odvodené priamo z údajov, odtiaľ termín "self-supervised". Medzi najznámejšie príklady patria:

1. **Predikcia rotácie obrázka:** Bežná úloha pretextu vo vizuálnom spracovaní. Obrázky sú otočené o preddefinovaný súbor uhlov a model musí predpovedať uhol rotácie. Táto úloha podnecuje model k učeniu sa vysokoúrovňových sémantických vlastností, ktoré sú invariantné voči rotácii.
2. **Farebnosť:** Modelu je daný obrázok v odtieňoch šedej a jeho úlohou je predpovedať pôvodné farby. Toto učí model rozumieť kontextu a obsahu obrázkov, čím ho podnecuje k učeniu sa vlastností súvisiacich s formami objektov a scénami.
3. **Riešenie skladačky Jigsaw:** Obrázky sú rozdelené na dlaždice, ktoré sú zamiešané. Úlohou modelu je rekonštruovať pôvodný obrázok predpovedaním správneho usporiadania dlaždíc. To pomáha modelu učiť sa o priestorových vzťahoch a kontexte v obrázku.
4. **Maskované modelovanie jazyka (MLM):** Používa sa v NLP, kde sú niektoré slová vo vete zakryté a model musí predpovedať zakryté slovo na základe okolitého kontextu. Táto úloha pomáha modelu vyvinúť hlboké porozumenie štruktúre a použitiu jazyka.

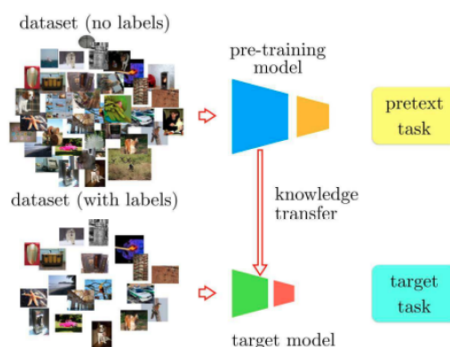


Obr. 2.8: Pretextové učenie [17]

2.4.8 Downstream Úlohy (Downstream Tasks)

[31][2] Downstream úlohy sú skutočné úlohy, pre ktoré je model v konečnom dôsledku určený. Ide o aplikácie v reálnom svete, ktoré ťažia z vlastností a reprezentácií naučených počas úlohy pretextu. Výkon na downstream úlohách je kľúčovým meradlom užitočnosti a generalizovateľnosti reprezentácií získaných prostredníctvom self-supervised učenia. Medzi možné príklady patria:

1. **Rozpoznávanie objektov:** Po tréningu na úlohe pretextu, ako je predikcia rotácie obrázka alebo riešenie skladačky, môže byť model jemne doladený na rozpoznávanie špecifických objektov na nových, neoznačených obrázkoch. Schopnosť rozpoznávať objekty v rôznych orientáciách a kontextoch je zlepšená vlastnosťami naučenými počas úlohy pretextu.
2. **Analýza sentimentu:** Model trénovaný pomocou maskovaného modelovania jazyka môže byť aplikovaný na analýzu sentimentu, kde potrebuje určiť sentiment vyjadrený v texte. Porozumenie nuanciám jazyka a kontextu získané z úlohy MLM pomáha v lepšom výkone na tejto downstream úlohe.
3. **Rozpoznávanie tvárí:** Ak bol model trénovaný na úlohe pretextu farebnosti, môže využiť naučené vlastnosti o ľudských tvárach a textúrach na zlepšenie výkonu v systémoch na rozpoznávanie tvárí.
4. **Rozpoznávanie reči:** Modely trénované na predpovedanie ďalšieho slova alebo doplnenie chýbajúcich slov v audio úlohách pretextu môžu byť prispôbené pre zložitejšie úlohy rozpoznávania reči, čím ťažia z nuancovaného porozumenia vzorom reči a kontextom.



Obr. 2.9: Downstream Task [17]

2.5 Strojové učenie, Deep learning a Neurónové siete

[8] Pojmy strojové učenie, deep learning často zvyknú byť používané zameniteľne a preto je dôležité všimnúť si jemný rozdiel medzi nimi. Strojové učenie, hlboké učenie a neurónové siete patria všetky do podoblasti umelej inteligencie, avšak neurónové siete sú podoblastou strojového učenia a hlboké učenie je podoblastou neurónových sietí.

Rozdiel medzi hlbokým učením a strojovým učením spočíva v spôsobe, akým sa každý algoritmus učí. Hlboké strojové učenie môže využívať označené dátové sady, známe ako

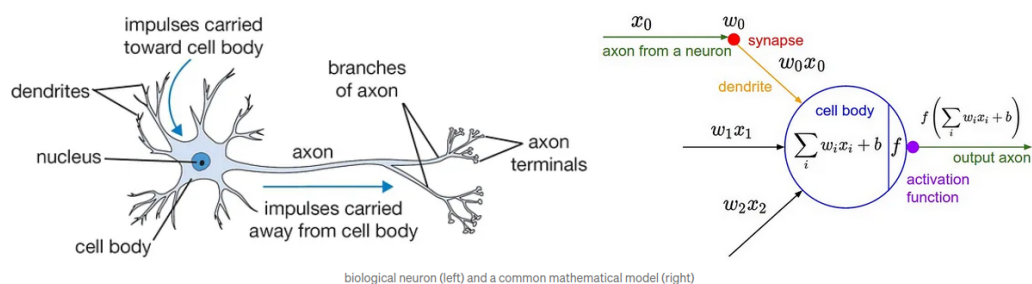
supervízované učenie 2.3, aby informovalo svoj algoritmus, ale nevyžaduje nutne označenú dátovú sadu. Proces hlbokého učenia môže prijímať neštruktúrované dáta vo svojej surovej forme (napr. text alebo obrázky) a automaticky určiť súbor vlastností, ktoré odlišujú rôzne kategórie dát od seba. Tým sa eliminuje niektorá z ľudskej intervencie, čo umožňuje použitie veľkého množstva dát. Hlboké učenie môžeme chápať ako "škálovateľné strojové učenie", ako poznamenáva Lex Fridman v tejto prednáške na MIT.

Klasické, alebo ne-hlboké, strojové učenie je viac závislé na ľudskej intervencii pre učenie. Ľudskí experti určujú súbor vlastností na pochopenie rozdielov medzi dátovými vstupmi, čo zvyčajne vyžaduje štruktúrovanejšie dáta pre učenie.

Neurónové siete 2.6, alebo umelé neurónové siete (ANNs), sú zložené z vrstiev uzlov, obsahujúcich vstupnú vrstvu, jednu alebo viac skrytých vrstiev a výstupnú vrstvu. Každý uzol, alebo umelý neurón, je spojený s iným a má priradenú váhu a prahovú hodnotu. Ak je výstup akéhokolvek individuálneho uzlu nad určenou prahovou hodnotou, tento uzol je aktivovaný a posielá dáta do ďalšej vrstvy siete. Inak, žiadne dáta nie sú posielané do ďalšej vrstvy siete týmto uzlom. "Hlboké" v hlbokom učení sa jednoducho vzťahuje na počet vrstiev v neurónovej sieti. Neurónová sieť, ktorá obsahuje viac ako tri vrstvy - vrátane vstupnej a výstupnej - môže byť považovaná za algoritmus hlbokého učenia alebo hlbokú neurónovú sieť. Neurónová sieť, ktorá má iba tri vrstvy, je len základná neurónová sieť.

2.6 Neurónove Siete

ň [6][4][11] Neurálne siete sú výpočtové modely inšpirované ľudským mozgom, ktoré sa skladajú z vrstiev prepojených uzlov (neurónov), ktoré spracúvajú informácie pomocou dynamických stavových odoziev na externé vstupy. Sú základom modernej umelej inteligencie, umožňujú vykonávať komplexné úlohy ako rozpoznávanie obrazu, spracovanie reči a rozhodovanie prostredníctvom adaptívneho učenia a schopnosti rozpoznávania vzorov. Tento prístup napodobňuje kognitívne funkcie, čo umožňuje strojom vykonávať úlohy, ktoré zvyčajne vyžadujú ľudskú inteligenciu.



Obr. 2.10: Porovnanie ľudského neurónu a matematického modelu neurónu [28]

Viacvrstvová neurónová sieť (FNN)

[27] Najjednoduchší typ neurónovej siete, kde spojenie medzi uzlami netvorí cyklus. Tento typ siete sa používa na asociácie medzi vstupnými údajmi a výstupmi, často slúži ako základ pre zložitejšie neurónové siete.

Konvolučná neurónová sieť (CNN)

[27] Hlavné používaná pri spracovaní údajov, ktoré majú mriežkovú topológiu, ako sú obrázky. CNN využívajú matematickú operáciu zvanú konvolúcia a sú mimoriadne účinné pri identifikácii vzorov a štruktúr v údajoch.

Rekurentná neurónová sieť (RNN)

[27] Navrhnutá na rozpoznávanie vzorov v sekvenciách údajov, ako sú texty, genómy alebo rukopisy. Výstupy z týchto sietí sú ovplyvnené predchádzajúcimi výpočtami a internou pamäťou, čo ich robí vhodnými pre úlohy, kde je kontext kľúčový.

Dlhodobá krátkodobá pamäť (LSTM)

[27] Špeciálny typ RNN schopný učiť sa dlhodobé závislosti. LSTM sú zvlášť užitočné pri učení sa z dôležitých udalostí s veľmi dlhými oneskoreniami medzi nimi v časových radách.

Jednotka s bránami rekurentná (GRU)

[27] Podobné ako LSTM s menej parametrami, čo ich robí efektívnejšími pri behu a stále schopnými učiť sa dlhodobé závislosti. Používajú sa tam, kde sú pamäť a výpočtový výkon obmedzené.

Autoenkódery

[27] Používané pre nesupervizované učenie efektívnych kódovaní, typicky pre účely zníženia dimenzionality alebo učenia sa funkcií. Autoenkódery sa učia kódovať vstup do menšieho zobrazenia a potom ho dekódovať späť na pôvodný vstup.

Generatívna súperivá sieť (GAN)

[27] Skladá sa z dvoch neurónových sietí, generátora a diskriminátora, ktoré súperia medzi sebou. Generátor vytvára čo najrealistickejšie údaje, a diskriminátor hodnotí ich pravosť. GANy sa široko používajú pri generovaní obrázkov, videí a hlasu.

Sieť s radiálnymi bazovými funkciami (RBF)

[27] Používa radiálne bazové funkcie ako aktivačné funkcie 2.7. Výstup siete je lineárnou kombináciou radiálnych bazových funkcií vstupov a parametrov neurónov. RBF siete sa používajú na aproximáciu funkcií, predikciu časových radov a riadenie.

Modulárna neurónová sieť (MNN)

[27] Skladá sa z kolekcie rôznych sietí, ktoré pracujú nezávisle a prispievajú k výstupu. Každá sieť v module vykonáva rôzne úlohy a ich výstupy sa kombinujú na efektívnejšie riešenie zložitých problémov.

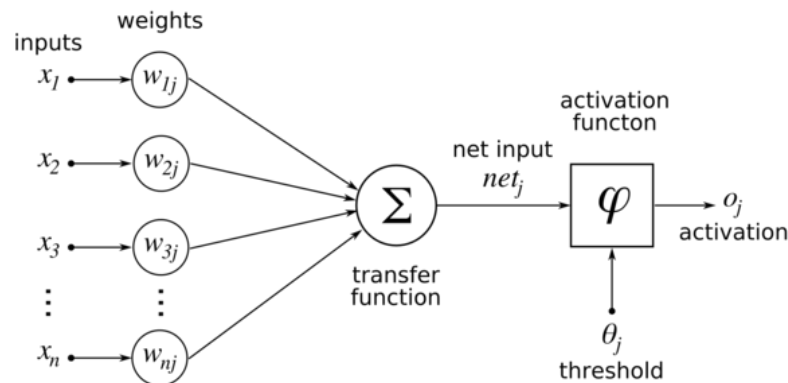
Siamská neurónová sieť

[27] Používa rovnaké váhy pri práci na dvoch rôznych vstupných vektoroch na výpočet porovnateľných výstupných vektorov. Sú obzvlášť užitočné pre úlohy, ktoré zahŕňajú hľadanie podobnosti alebo vzťahu medzi dvoma porovnateľnými vecami, ako je overovanie tváre.

2.7 Aktivačné funkcie

[22][23][31] Neurónové siete 2.6 sú zložené z neurónov, ktoré prijímajú numerické vstupy. Každý neurón v sieti je vážený a tieto váhy sa násobia s vstupmi pre získanie výstupu, ktorý je ďalej poslaný do nasledujúcej vrstvy. Aktivačná funkcia v neurónoch funguje ako matematický prekladač, ktorý transformuje vstupné signály do výstupov prenosných do ďalších vrstiev.

Aktivačné funkcie sú základnou súčasťou neurónových sietí, ktoré rozhodujú, či má byť neurón aktivovaný alebo nie. Stanovením výstupu neurónov, tieto funkcie pridávajú nelineárne vlastnosti do siete, čo zvyšuje jej schopnosť učiť sa zložené vzory údajov.



Obr. 2.11: Princíp aktivačných funkcií v neuronových sieťach [26]

Typy Aktivačných Funkcií

- Binárna Skoková Funkcia - Tento typ aktivačnej funkcie aktivuje alebo deaktivuje výstup neurónu na základe preddefinovanej prahovej hodnoty.
- Lineárna Aktivačná Funkcia - Proporciónálne prenáša vstup na výstup, používa sa menej často kvôli svojej obmedzenej schopnosti modelovania nelineárnych vzťahov.
- Nelineárna Aktivačná Funkcia - Umožňuje neurónovej sieti vykonávať komplexné mapovania medzi vstupmi a výstupmi. Tieto funkcie sú nevyhnutné pre modelovanie komplexných dát ako obrázky alebo zvuky.

Nelineárne aktivačné funkcie majú niekoľko dôležitých výhod:

1. Umožňujú efektívne spätné šírenie chýb v procese učenia (backpropagation).
2. Podporujú konštrukciu hlbokých neurónových sietí s mnohými vrstvami, čo je kritické pre spracovanie a učenie z komplexných súborov údajov.

Nelineárne aktivačné funkcie sú kľúčové pre pokročilé aplikácie neurónových sietí, umožňujúce modelovanie zložitých vzorcov a predikcií potrebných v mnohých moderných technologických aplikáciách.

2.7.1 Sigmoidná Funkcia

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.6)$$

Sigmoidná funkcia dáva výsledky medzi 0 a 1, čo je užitočné pre problémy binárnej klasifikácie. Tradične sa používa na výstupnej vrstve binárnych klasifikátorov pre mapovanie hodnôt na pravdepodobnosti. Trpí problémom miznúceho gradientu, kde gradienty sa stávajú veľmi malými, efektívne brániaci sieť v ďalšom učení počas spätnej propagácie, keď hodnoty gradientu dosiahnu nulu v hlbších vrstvách.

2.7.2 Hyperbolická Tangentná Funkcia (Tanh)

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.7)$$

Tanh tiež dáva výsledky v rozsahu od -1 do 1. Je podobná sigmoidnej, ale je centrová okolo nuly, čo ju robí vo všeobecnosti lepšou pre skryté vrstvy, pretože v niektorých prípadoch vedie k rýchlejšej konvergencii. Podobne ako sigmoid 2.7.1, tanh tiež trpí problémom miznúceho gradientu.

2.7.3 ReLU (Rectified Linear Unit)

ReLU je skratka pre *Rectified Linear Unit*, je to typ aktivačnej funkcie používaný hlavne v hlbokom učení a umelých neurónových sieťach. Je definovaný takto:

$$\text{ReLU}(x) = \max(0, x) \quad (2.8)$$

ReLU sa stalo predvolenou aktivačnou funkciou pre mnoho typov neurónových sietí, pretože umožňuje modelom konvergovať rýchlo. Rieši problém miznúceho gradientu, umožňuje modelom učiť sa rýchlejšie a efektívnejšie. ReLU má však aj určité nevýhody, ako napríklad problém „umierajúcich ReLU.“ Ten nastáva, keď veľká časť neurónov stále vracia nulu, čím sa v zablokujú a nikdy sa neaktivujú počas tréningu.

2.7.4 Leaky ReLU

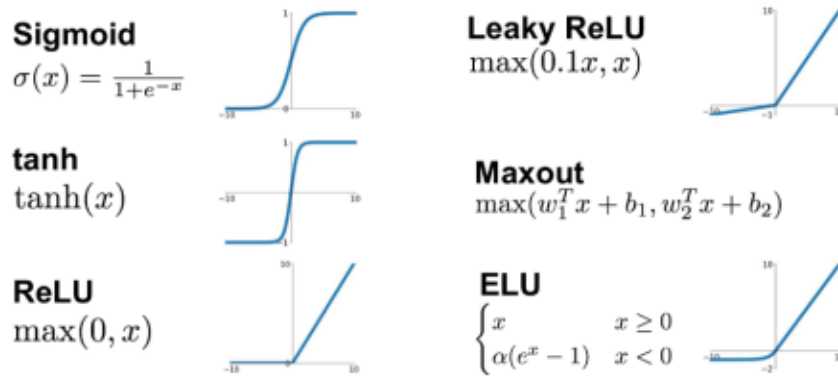
$$\text{Leaky ReLU}(x) = \max(0.01x, x) \quad (2.9)$$

Táto funkcia sa snaží vyriešiť problém „umierajúcich neurónov“ v ReLU tým, že umožňuje malý, pozitívny gradient, keď je jednotka neaktívna. Poskytuje všetky výhody ReLU, plus neexistuje riziko mŕtvych neurónov, pretože umožňuje malý gradient, keď jednotka nie je aktívna.

2.7.5 Softmax

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_k e^{x_k}} \quad (2.10)$$

Softmax funkcia je zovšeobecnením sigmoidnej funkcie 2.7.1 na viaceré triedy. Používa sa na výstupnej vrstve viactriednej klasifikačnej neurónovej siete. Vracia pravdepodobnosti každej triedy, pričom cieľová trieda má najvyššiu pravdepodobnosť. Typicky sa používa pre problémy viactriednej klasifikácie.



Obr. 2.12: Príklady aktivačných funkcií [21]

Aktivačné funkcie hrajú kľúčovú úlohu v neurónových sieťach tým, že zavádzajú nelineárne vlastnosti, ktoré umožňujú sieti učiť sa zložité vzory v údajoch. Výber aktivačnej funkcie môže výrazne ovplyvniť výkonnosť a dynamiku tréningu neurónovej siete.

2.8 Optimizátory

[4][25] Optimalizácia parametrov modelu je kľúčovým aspektom tréningu algoritmov strojového učenia. Medzi tri základné skupiny optimalizačných techník patria: Gradientový zostup, Stochastický gradientový zostup, a pokročilé optimalizátory ako Adam a RMSprop. Tieto metódy sa využívajú na minimalizáciu chybovej funkcie a zlepšenie výkonnosti modelov na rôznych úlohách strojového učenia. Optimalizácia je základným pilierom úspešného strojového učenia.

2.8.1 Gradientový zostup

[24] Gradientový zostup je základná metóda optimalizácie používaná na minimalizáciu chybovej funkcie modelu v strojovom učení. Základný princíp spočíva v iteratívnej aktualizácii parametrov modelu v smere opačnom k gradientu chybovej funkcie. Tento prístup umožňuje postupné znižovanie hodnoty chyby a konvergenciu k lokálnemu alebo globálnemu minimu.

Parametre modelu sú aktualizované nasledujúcim spôsobom:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta) \quad (2.11)$$

kde θ sú parametre modelu, η je rýchlosť učenia a $\nabla_{\theta} J(\theta)$ je gradient chybovej funkcie vzhľadom na parametre.

Varianty

- **Batch Gradient Descent:** Vypočíta gradient chybovej funkcie na celom datasete, čo zaručuje stabilný smer k optimu ale vyžaduje veľké množstvo výpočtov.
- **Stochastic Gradient Descent (SGD):** Vypočíta gradient na základe jedného príkladu, čo umožňuje rýchlejšie iterácie a môže pomôcť uniknúť z lokálnych miním.
- **Mini-batch Gradient Descent:** Kompromis medzi predchádzajúcimi dvoma prístupmi, kde gradient je počítaný z malej skupiny vzoriek.

2.8.2 Stochastický gradientový zostup (SGD)

[24] Stochastický gradientový zostup je rozšírením gradientového zostupu, ktorý aktualizuje parametre modelu po každom jednotlivom príklade v tréningovom datasete. Tento prístup znižuje pamäťové požiadavky a umožňuje rýchlejšiu konvergenciu, čo je výhodné pri veľkých datasetoch. SGD je obľúbená varianta gradientového zostupu vďaka svojej efektívnosti pri práci s veľkými dátami. Aktualizácia parametrov modelu prebieha ako:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)}) \quad (2.12)$$

kde $x^{(i)}, y^{(i)}$ sú náhodne vybrané príklady z tréningového datasetu.

2.8.3 Adam (Adaptive Moment Estimation)

[15] Adam je pokročilá optimalizačná metóda, ktorá kombinuje výhody momentum a RMS-prop techník. Adam adaptívne upravuje rýchlosti učenia pre jednotlivé parametre na základe odhadov prvých a druhých momentov gradientov.

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} J(\theta) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} J(\theta))^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{aligned}$$

kde β_1, β_2 sú koeficienty zanikania pre momenty a ϵ je konštanta zabráňujúca deleniu nulou.

2.8.4 RMSprop (Root Mean Square Propagation)

[22] RMSprop je optimalizačná metóda, ktorá upravuje rýchlosť učenia pre každý parameter zvlášť na základe priemeru nedávnych štvorcov gradientov.

$$v_t = \beta v_{t-1} + (1 - \beta) (\nabla_{\theta} J(\theta))^2 \quad (2.13)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla_{\theta} J(\theta) \quad (2.14)$$

kde v_t je kumulatívny priemer štvorcov gradientov a β je koeficient zanikania.

2.9 Funkcie straty

[19][4] Funkcie straty sú kľúčovým prvkom v procese tréningovania modelov v strojovom učení, ktoré umožňujú kvantifikovať rozdiel medzi skutočnými a predikovanými hodnotami. Medzi najpoužívanjšie patria najmä:

2.9.1 Mean Squared Error (MSE)

[19][4] MSE je často používaná v regresných úlohách na kvantifikáciu rozdielu medzi predikovanými a skutočnými hodnotami. Funkcia MSE vypočíta priemer štvorcov rozdielov

medzi predikovanými a skutočnými hodnotami. Cieľom je minimalizovať tento rozdiel, čo vedie k lepšej presnosti predikcií. Pre binárnu klasifikáciu: Mean Squared Error je definovaný ako:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

kde y_i sú skutočné hodnoty, $\hat{y}_i$ sú predikované hodnoty a n je počet vzoriek.

2.9.2 Cross-Entropy Loss

[19][4] Cross-Entropy Loss sa často používa v klasifikačných úlohách na kvantifikáciu rozdielu medzi distribúciou tried predikovanou modelom a skutočnou distribúciou tried. Táto funkcia straty využíva princíp maximálnej pravdepodobnosti. Vypočíta sa logaritmus pravdepodobnosti predikcie každej triedy a následne sa spočíta priemerná hodnota týchto logaritmických pravdepodobností. Cieľom je minimalizovať hodnotu tejto funkcie, čo znamená, že model lepšie aproximuje skutočnú distribúciu tried.

Pre binárnu klasifikáciu:

$$\text{Cross-Entropy} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Pre viactriednu klasifikáciu:

$$\text{Cross-Entropy} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$$

kde y_{ic} je indikátorová premenná pre prítomnosť triedy c pre i -tú vzorku, $\hat{y}_{ic}$ je predpovedaná pravdepodobnosť, že i -tá vzorka patrí do triedy c , a C je počet tried.

2.9.3 Hinge Loss

[19][4] Hinge Loss sa často používa v klasifikácii s support vector machines (SVM) na binárne klasifikačné úlohy. Táto funkcia straty penalizuje nesprávne klasifikované príklady a podporuje vytvorenie širokého okraja medzi triedami. Je definovaná ako maximálna hodnota z nuly a rozdielu medzi správnou triedou a najväčšou inou triedou.

$$\text{Hinge Loss} = \max(0, 1 - y_i \cdot \hat{y}_i)$$

kde y_i sú skutočné triedy so značkami -1 alebo 1 a $\hat{y}_i$ sú predikované hodnoty.

2.9.4 Kullback-Leibler Divergence

[19][4] Kullback-Leibler Divergence sa používa v rámci porovnávania dvoch pravdepodobnostných distribúcií. Táto funkcia straty môže byť použitá na meranie podobnosti medzi dvoma distribúciami pravdepodobnosti. Je definovaná ako rozdiel medzi očakávanou hodnotou logaritmu pravdepodobnosti v skutočnej distribúcii a v predikovanej distribúcii.

$$\text{KL Divergence} = \sum_i p(i) \log \left(\frac{p(i)}{q(i)} \right)$$

kde $p(i)$ je pravdepodobnosť skutočnej distribúcie pre príklad i a $q(i)$ je pravdepodobnosť predpovedanej distribúcie pre príklad i .

2.10 Hodnotiace metriky

[4] Hodnotiace metriky sú dôležitým nástrojom na posúdenie výkonnosti modelov v strojovom učení. Medzi rôzne typy hodnotiacich metrík patria:

2.10.1 Accuracy

Accuracy je jedným z najbežnejších metrík na vyhodnocovanie klasifikačných modelov. Vyjadruje percentuálny podiel správne klasifikovaných príkladov voči celkovému počtu príkladov. Vyššia hodnota presnosti naznačuje lepší výkon modelu.

2.10.2 Precision) a Recall

Precision a Recall sú často používané metriky v klasifikačných úlohách s nevyváženými triedami. Precision vyjadruje podiel správne klasifikovaných pozitívnych príkladov medzi všetkými pozitívnymi predikciami, zatiaľ čo Recall vyjadruje podiel správne klasifikovaných pozitívnych príkladov medzi všetkými skutočnými pozitívnymi príkladmi.

2.10.3 F1-Score

F1-Score je harmonický priemer metrík Precision a Recall a používa sa na vyhodnocovanie výkonu modelov v klasifikačných úlohách s nevyváženými triedami. Je to vyvážená metrika, ktorá berie do úvahy ako presnosť, tak aj úplnosť.

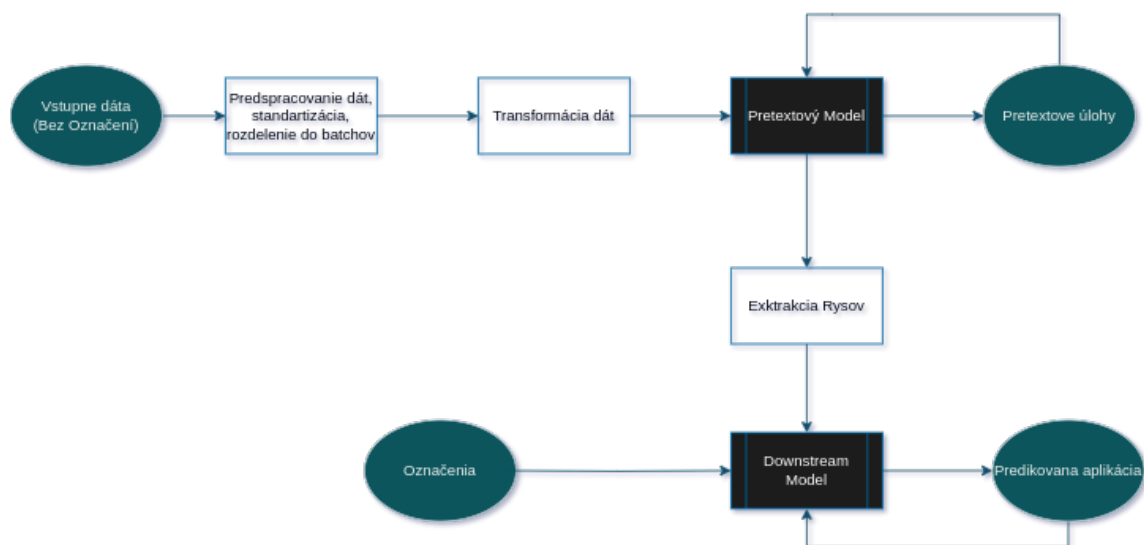
2.10.4 ROC krivka (Receiver Operating Characteristic curve) a ROC AUC (Area Under the Curve)

ROC krivka a ROC AUC sa často používajú na vyhodnocovanie binárnych klasifikačných modelov. ROC krivka je grafická reprezentácia závislosti medzi True Positive Rate (Recall) a False Positive Rate. ROC AUC je plocha pod ROC krivkou a vyjadruje celkovú schopnosť modelu oddeliť medzi pozitívnymi a negatívnymi príkladmi.

Kapitola 3

Návrh Implementácie

Cieľom tejto bakalárskej práce je preskúmať a rozšíriť možnosti štatistickej analýzy na úrovni sieťových tokov prostredníctvom využitia techník self-supervised učenia. Práca sa zameriava na analýzu typických charakteristík využívaných pri analýze sieťových tokov, aby sa pochopili základné vzory a anomálie v dátach sieťového toku. Ďalej navrhne a implementuje nový model pre extrakciu reprezentácie dát v sieťových tokoch, ktorý bude využívať pokročilé metódy učenia na autonómne učenie sa užitočných vlastností z neoznačených sieťových dát. V rámci práce je plánom vyskúšať viacero pretextových úloh self-supervised learning (SSL), a porovnať ich výkonnosť na downstream úlohe, aby sa identifikovali najefektívnejšie metódy pre modelovanie sieťových dát. Model bude experimentálne implementovaný a natrénovaný s použitím knižnice PyTorch, aby sa zabezpečilo, že efektívne sa učí z dát sieťového toku bez potreby externého dohľadu alebo označených dát. Práca tiež zahŕňa použitie datasetu sieťových služieb na posúdenie kvality a efektivity reprezentácií získaných vyvinutým modelom a diskusiu o výkonnosti modelu v rôznych testoch a scenároch na pochopenie jeho silných stránok a obmedzení. Ďalej sa práca bude zameriavať na preskúmanie vplyvu na výkonnosť a presnosť modelu jednotlivých pretextových úloh a metódy kontrastného učenia, pomocou menenia váh jednotlivých stratových funkcií pretextových úloh v kombinovaných modeloch. Nasledná extrakcia reprezentácií a vyhodnotenie ich na downstreamovom tasku pre zistenie ich efektivity bude kľúčovou súčasťou práce na zistenie najoptimálnejšej konfigurácie pre efektívnu analýzu. Na základe týchto poznatkov bude navrhnuté potenciálne budúce rozšírenie a aplikácie modelu v oblasti bezpečnosti siete a analýzy toku. Táto práca má ambíciu významne prispieť k technologickému pokroku v analýze sieťového toku, poskytujúc robustný rámec, ktorý môže zlepšiť detekciu anomálií, monitorovanie systému a opatrenia kybernetickej bezpečnosti v komplexných sieťových prostrediach.



Obr. 3.1: Návrh architektury riešenia

3.1 Výber Technológií

Pri výbere technológií pre implementáciu projektu boli kľúčové nasledujúce aspekty: flexibilita, podpora širokej komunity, možnosti integrácie a efektivita pri spracovaní dát. Tieto kritéria ma viedli k výberu nasledujúcich nástrojov a knižníc:

3.1.1 PyTorch

[1]PyTorch je jednou z najpopulárnejších knižníc pre hlboké učenie, ktorá je známa svojou modularitou, efektivitou a dynamickým grafom, čo umožňuje flexibilné experimentovanie s modelmi hlbokého učenia. Tieto vlastnosti sú ideálne pre môj projekt, kde je potreba častého testovania a upravovania modelov v reakcii na získané výsledky z experimentov. Flexibilná architektúra PyTorch podporuje rýchlu iteráciu a testovanie rôznych prístupov, čo je kritické pre úspech mojej práce na extrakcii reprezentácií dát. Navyše, PyTorch poskytuje vynikajúcu podporu pre prácu s tenzormi, čo sú základné stavebné kamene v hlbokom učení. Knižnica umožňuje efektívne manipulácie s tenzormi a ich automatické diferencovanie, čo je zásadné pre tréning modelov strojového učenia. Vďaka optimalizovanému výkonu a podpore rôznych hardvérových platform vrátane GPU, PyTorch uľahčuje rýchle a efektívne spracovanie veľkých objemov dát, čo je nevyhnutné pre efektívne učenie modelov z rozsiahlych neoznačených datasetov. Tieto aspekty robia PyTorch ideálnym nástrojom pre moju prácu a umožňujú mi vyvinúť robustné riešenia pre analýzu sieťových tokov.

3.1.2 CESNET DataZoo

[5] CESNET DataZoo je komplexná sada nástrojov navrhnutá na pomoc výskumníkom a vývojárom pri správe veľkých datasetov sieťovej prevádzky, najmä pre experimenty s klasifikáciou prevádzky. Poskytuje štandardné API na prístup a manipuláciu s tromi kľúčovými datasetmi: CESNET-TLS22, CESNET-QUIC22 a CESNET-TLS-Year22, ktoré obsahujú šifrované dáta sieťovej prevádzky zozbierané počas rôznych časových období. Kľúčové funkcie CESNET DataZoo zahŕňajú:

- Prístup k datasetom a ich konfigurácia: Uľahčuje sťahovanie, konfiguráciu a načítanie datasetov, čo umožňuje definovať trénovacie, validačné a testovacie obdobia. Táto flexibilita podporuje časovo závislé hodnotenie, ktoré je kľúčové pre pochopenie, ako modely fungujú počas rôznych období a za rôznych podmienok.
- Spracovanie dát: Nástroj podporuje spracovanie dávok prostredníctvom PyTorch DataLoaders aj načítanie celého datasetu do pamäte pomocou Tenosrov. To je obzvlášť užitočné pri manipulácii s veľkými datasetmi, ktoré sa nezmestia naraz do pamäti.

3.1.3 Jupyter Notebook a Vzdialený Počítač

[14]Projekt je implementovaný v prostredí Jupyter Notebook, ktorý je spustený na vzdialenom počítači s vysokým výpočtovým výkonom. Toto prostredie mi umožňuje pohodlnú a interaktívnu prácu s kódom, zatiaľ čo vzdialený počítač poskytuje potrebné zdroje pre rýchle spracovanie veľkého množstva dát, čo je nevyhnutné pre tréning náročných modelov hlbokého učenia.

3.1.4 Dodatočné Knižnice

Pre vizualizáciu výsledkov a metrík používam knižnice ako `matplotlib.pyplot` a `sklearn.metrics`. Matplotlib je široko používaná knižnica pre vykresľovanie grafov, ktorá umožňuje jednoduché a efektívne vizualizácie experimentálnych výsledkov, zatiaľ čo confusion matrix z knižnice Scikit-learn je nástroj pre hodnotenie klasifikačných modelov, čo je kľúčové pre analýzu výkonnosti navrhovaných modelov.

3.2 Vstupné Dáta

3.2.1 CESNET TLS22

[3][16] Nedávny úspech a rozšírenie strojového učenia a hlbokého učenia poskytli silné nástroje, ktoré sa využívajú aj na analýzu, klasifikáciu a detekciu hrozieb v šifrovanej sieťovej prevádzke. Keďže väčšina sieťovej prevádzky je šifrovaná, tradičné riešenia založené na hlbkovej inšpekcii paketov (DPI) strácajú na význame, a preto je naliehavá potreba moderných klasifikačných metód schopných analyzovať šifrovanú prevádzku. Tieto metódy sa musia vzdať obsahu balíkov a sústrediť sa na štatistiky tokov a metadáta paketov, ako sú veľkosti paketov, smerovanie a časy príchodu.

Dataset CESNET-TLS22 je v tomto kontexte mimoriadne relevantný, pretože poskytuje obrovský objem dát potrebných na trénovanie týchto komplexných neurónových modelov. S 141 miliónmi záznamov tokov a podrobnými metadátami podporuje vývoj a testovanie strojových učiacich sa modelov schopných klasifikovať šifrovanú prevádzku. Navyše, výzva odmietnuť alebo správne zaobchádzať s neznámou prevádzkou, ktorá nebola počas trénovacej fázy viditeľná, je oblasťou, v ktorej môžu neurónové siete vyniknúť vďaka ich schopnosti generalizovať z veľkých datasetov.

Poskytnutím takého bohatého a rozsiahleho datasetu umožňuje CESNET-TLS22 výskumníkom a vývojárom účinne riešiť výzvy klasifikácie šifrovanej prevádzky a detekcie neznámej, potenciálne škodlivej prevádzky. To ho robí neoceniteľným zdrojom pre pokrok v schopnostiach kybernetickej bezpečnosti v ére, kde je šifrovanie prítomné takmer všade.

Format dát

Formát dát v datasete CESNET-TLS22 je navrhnutý tak, aby poskytoval bohaté informácie vhodné pre analýzu šifrovanej sieťovej prevádzky pomocou moderných metód strojového učenia. Dáta sú štruktúrované v komprimovaných CSV súboroch, kde každý riadok predstavuje jeden sieťový tok. Pre každý tok dataset obsahuje nasledujúce atribúty:

- **ID**: Jedinečný identifikátor
- **BYTES**: Počet bajtov prenesených od klienta k serveru
- **BYTES_REV**: Počet bajtov prenesených od serveru ku klientovi
- **PACKETS**: Počet paketov odoslaných od klienta k serveru
- **PACKETS_REV**: Počet paketov odoslaných od serveru ku klientovi
- **DURATION**: Trvanie toku v sekundách
- **PPI** (Packet Profile Information): Sekvencia metadát paketu vo formáte, ktorý zahŕňa medzipríchodové časy, smery paketov a veľkosti paketov
- **PPI_LEN**: Počet paketov v sekvencii PPI
- **PPI_DURATION**: Trvanie sekvencie PPI v sekundách
- **PPI_ROUNDTRIPS**: Počet roundtripov (od a do cieľa a späť) v sekvencii PPI
- **APP**: Označenie webovej služby
- **CATEGORY**: Kategória služby
- **TCP_FLAGS**: TCP príznaky odoslané od klienta k serveru
- **TCP_FLAGS_REV**: TCP príznaky odoslané od serveru k klientu

Pre moju prácu, sa primárne sústredim na hodnoty PPI, t.j. paketové veľkosti, paketové smery a medzipríchodové časy, je tento dataset ideálny. Hodnoty PPI poskytujú dôležité informácie o charakteristikách sieťového toku, ktoré môžu byť použité na tréning modelov na detekciu anomálií alebo klasifikáciu sieťovej prevádzky bez potreby rozbalenia obsahu paketov. Tieto údaje umožňujú modelom strojového učenia zachytiť vzorce správania v šifrovanej komunikácii, čo je kľúčové v súčasnom prostredí, kde je veľa prevádzky šifrovanej.

3.2.2 Práca s dátami

V mojej bakalárskej práci používam CESNET DataZoo na načítavanie dát pre analýzu šifrovanej sieťovej prevádzky pomocou datasetu CESNET-TLS22. Tento proces je kľúčový pre efektívne spracovanie a analýzu dát, pričom sa zameriavam na najrelevantnejšie informácie dostupné v datasete.

Načítavanie Dát

Dataset CESNET-TLS22 je načítaný s využitím konfigurácie, ktorá zohľadňuje potreby mojich výskumných aktivít. Vyberám len dáta z desiatich najznámejších internetových aplikácií, čím sa sústredím na dominantné a najčastejšie používané služby. Tento prístup umožňuje presnejšie a relevantnejšie výsledky v oblasti detekcie anomálií a klasifikácie prevádzky.

Výber Dátového Obdobia

Dáta pochádzajú z dvoch špecifických týždňov v roku 2021 (40. a 41. týždeň), čo zaisťuje, že analyzované informácie odrážajú aktuálne vzorce správania v sieťovej komunikácii.

Normalizácia Dát

Pri analýze a spracovaní dát využívam MinMax škálovanie (ScalerEnum.MINMAX) pre rôzne metriky, ako sú veľkosti paketov, časy medzi paketmi a štatistiky tokov. Normalizácia zabezpečuje, že vstupy do modelov strojového učenia sú uniformné, čo je kritické pre konzistentné a efektívne učenie.

Dávkovanie a Rozporcovanie Dát

Trénovací DataLoader je nastavený na náhodné poradie, aby sa predišlo prílišnému prispôbeniu modelu na konkrétne vzory v tréningových dátach, čím sa zvyšuje jeho schopnosť generalizácie. Veľkosť dávkovania je stanovená na 32 príkladov na dávku, čo umožňuje efektívne učenie bez nadmerného zataženia systémových zdrojov.

Rozdelenie Dát

Dáta sú rozdelené do troch hlavných skupín: tréningové, validačné a testovacie DataLoadery. Toto rozdelenie umožňuje systematické testovanie a overovanie modelu v rôznych fázach jeho vývoja - od počiatočného tréningovania, cez validáciu až po konečné testovanie na nevidených dátach. Rozdelenie dát na tréningové, validačné a testovacie súbory je nevyhnutné pre objektívne hodnotenie účinnosti a presnosti navrhovaných modelov.

3.3 Architektúra Modelov

V rámci mojej práce plánujem navrhnuť a vyvinuť tri rôzne modely strojového učenia, v ktorých porovnam vplyv rozličných pretextových úloh. Tento experimentálny prístup umožní hlbšie porozumenie, ako rôzne metódy self-supervised learning (SSL) ovplyvňujú schopnosť modelov naučiť sa užitočné reprezentácie pre klasifikačné úlohy.

3.3.1 Prvý Model - Maskovanie vs Premiešavanie PPI

V tomto modeli budem porovnávať účinnosť modelu, ktorý používa pretextovú úlohu maskovania, proti modelu, ktorý používa premiešavanie packet profile information (PPI). Maskovacia úloha zahŕňa skrývanie častí dát (napríklad určitých metadát paketov) a núti model predpovedať tieto skryté informácie. Na druhej strane, premiešavanie PPI zamieňa poradie paketov alebo ich časti, čím model musí nájsť správne poradie alebo vzťahy medzi časťami.

3.3.2 Druhý Model - Maskovanie vs Kontrastné Učenie

Druhý model skúma rozdiely medzi maskovaním dát a kontrastným učením, ktoré využíva maskované dáta. Kontrastné učenie v tomto prípade spočíva v rozlišovaní medzi správnymi a nesprávnymi verziami dát, čo by malo viesť k efektívnejšiemu učeniu sa rozdielov medzi klasami alebo charakteristikami dát.

3.3.3 Tretí Model - Klasifikačná Úloha vs Kontrastné Učenie s Označeniami

V treťom modeli porovnáam priamočiare klasifikačné učenie s kontrastným učením, kde sa používajú labely z datasetu na naučenie modelu rozlišovať medzi rôznymi triedami. Kontrastné učenie v tomto prípade bude využívať explicitné labely na posilnenie modelu, aby lepšie rozumel rozdielom a podobnostiam medzi jednotlivými triedami.

3.3.4 Spoločná Architektúra a Evaluácia Modelov

Každý z týchto modelov bude navrhnutý tak, aby z jednotlivých prístupov extrahoval reprezentácie dát, ktoré budú následne evaluované na downstream úlohe klasifikácie aplikácií. Hlavnými metrikami na evaluáciu účinnosti modelov budú presnosť (accuracy), recall, presnosť predikcie (precision), F1 skóre a matica zámien (confusion matrix) na vizualizáciu výsledkov. Tieto metriky umožnia detailné porovnanie toho, ako dobre modely rozpoznávajú a klasifikujú rôzne typy sieťovej prevádzky na základe naučených reprezentácií.

Kapitola 4

Implementácia

4.1 Pretextové úlohy

V rámci mojej implementácie modelov som použil nasledovné pretextové úlohy a metódy v kontexte self-supervised learning (SSL):

4.1.1 Pretextová Úloha: Zamiešanie

V úlohe zamiešania sú sekvencie dát paketov zahŕňajúce intervaly paketov, smer a veľkosti rozdelené na segmenty a náhodne zamiešané. Model sa učí obnoviť pôvodný poriadok týchto segmentov. Toto učí model porozumieť sekvencii a kontextu, čo je dôležité v scenároch, ako je detekcia anomálií, kde odchýlky od typických vzorcov paketov môžu naznačovať bezpečnostné hrozby. Premiešavanie PPI vytvára výzvu pre model, aby sa naučil rozpoznávať a obnovovať pôvodné vzory a štruktúry dát z náhodne zmiešaných vstupov. Táto schopnosť je obzvlášť cenná pri analýze sieťových tokov, kde môžu byť dáta často fragmentované alebo poškodené. Premiešavanie teda poskytuje robustný test schopnosti modelu adaptovať sa na neusporiadané alebo poškodené dáta a zároveň zachovať svoju schopnosť predikcie alebo klasifikácie.

4.1.2 Pretextová Úloha: Maskovanie

Pretextová úloha maskovania spočíva v náhodnom maskovaní určitého percenta hodnôt v sekvenciách PPI (Packet Profile Information), ktoré obsahujú dôležité metadáta o sieťových tokoch, ako sú veľkosti paketov, smery paketov a medzipríchodové časy. Vstupom sú teda PPI hodnoty a maskovacie percento je nastavené na 0.15.

Cieľom tohto prístupu je simulovať scenáre, kde model musí predpovedať alebo doplniť chýbajúce informácie, čo je zásadné pre jeho schopnosť generalizácie a robustnosti voči neúplným dátam. Tento druh úlohy pomáha modelu lepšie pochopiť a identifikovať podstatné vzorce v dátach, čo je kľúčové pre jeho úspešné nasadenie v reálnych aplikáciách, kde často dochádza k strate alebo poškodeniu dát.

4.1.3 Kontrastné učenie

Táto metóda používa páry dát, kde sú niektoré páry označené ako podobné a iné ako rozdielne. Model sa učí rozlišovať medzi týmito dvoma kategóriami a extrahovať charakteristiky, ktoré identifikujú podobnosti alebo rozdiely medzi párovými prvkami. Tento prístup

je obzvlášť užitočný pre zlepšenie rozlišovacej schopnosti modelu a pre zlepšenie jeho generalizačných vlastností.

V kontexte môjho modelovania som využil kontrastný loss, špecificky InfoNCE loss, ktorý je zásadný pre účinné učenie pomocou kontrastného učenia.

Pred aplikáciou InfoNCE loss je kľúčové normalizovať vektory vloženia (embeddings). To zaisťuje, že všetky vloženia majú jednotkovú normu, čo vedie k stabilnejšiemu a konzistentnejšiemu výpočtu podobnosti medzi vektormi. Normalizácia sa vykonáva pomocou L2 normalizácie, ktorá delí každý vektor jeho normou L2 plus veľmi malou hodnotou (napríklad $1e-12$), aby sa predišlo deleniu nulou. InfoNCE loss je formulovaný tak, aby zvýraznil podobnosť medzi "kotvovými" vloženími a kladnými vloženími, ktoré by mali byť podobné, a zároveň maximalizoval rozdiel medzi kotvovými vloženími a negatívnymi vloženími, ktoré by mali byť odlišné. Tento loss využíva koncept teploty (temperature), ktorý reguluje ostré rozdelenie podobností, čím umožňuje lepšie rozlíšenie medzi podobnými a nepodobnými pármami.

Tento prístup posilňuje model tým, že ho núti rozlišovať medzi skutočne podobnými a nepodobnými vzormi, čo vedie k efektívnemu učeniu robustných reprezentácií vhodných pre širokú škálu aplikácií. InfoNCE loss je široko používaný v modeloch, kde je cieľom naučiť sa z neoznačených dát a je ideálny pre scenáre, kde sú označené dáta obmedzené alebo nedostupné.

4.2 Funkcia na extrakciu dát

Kazdy model v mojej bakalárskej práci zahŕňa aj funkciu na extrakciu reprezentácií, ktorá je kľúčová pre neskoršie využitie v aplikovaných úlohách, ako je klasifikácia. Táto funkcia umožňuje modelu zachytiť a vyčleniť dôležité vlastnosti z dát, ktoré môžu byť následne použité na posilnenie výkonu pri klasifikačných úlohách.

Extrakcia reprezentácií je proces, pri ktorom model analyzuje a transformuje vstupné dáta do formy, ktorá lepšie sumarizuje podstatné informácie obsiahnuté v dátach.

Funkcionalita extrakcie reprezentácií poskytuje viaceré výhody:

- Zvýšenie presnosti modelu: Efektívne sumarizácia a reprezentácia dát môže výrazne zlepšiť schopnosť modelu presne klasifikovať alebo predpovedať výstupy, pretože model má lepší prehľad o podstatných vzoroch a trendoch v dátach.
- Zníženie dimenzionality: Redukcia dimenzionality dát do užitočných reprezentácií môže pomôcť zefektívniť výpočty a zjednodušiť modely, čím sa znižuje riziko preučenia a zlepšuje sa rýchlosť tréningu a inferencie.
- Využitie v rôznych aplikáciách: Reprezentácie extrahované modelom môžu byť použité v rôznych downstream úlohách, čím sa rozširuje možnosť aplikácie modelu na rôzne dátové sady alebo problémy.

4.3 Model 1 - Kombinovaný SSL Model

4.3.1 Architektúra Modelu

V rámci mojej bakalárskej práce som sa rozhodol použiť sieť Long Short-Term Memory (LSTM) na spracovanie sekvencií dát, ktoré zahŕňajú dve rôzne pretextové úlohy – jednu

na predikciu správneho poradia segmentov a druhú na predpovedanie zamaskovaných hodnôt. Tieto siete boli vybrané kvôli ich schopnosti zachytávať dlhodobé závislosti a vzory v časových radoch, čo je kľúčové pre úspešné spracovanie komplexných sekvencií, ako sú PPI (Packet Profile Information) sekvencie.

LSTM siete sú ideálne pre úlohy, kde je potrebné spracovávať dáta s vysokou úrovňou nepravidelnosti a neúplnosti, pretože dokážu efektívne spracovávať vstupy rôznych dĺžok a sú robustné voči rušeniu v dátach. Model v mojej práci zahŕňa dve hlavné komponenty: LSTM maskovacia vrstva a LSTM vrstva pre korekciu poradia.

LSTM maskovacia vrstva sa zaoberá spracovaním zamaskovaných dát, kde je cieľom predpovedať pôvodné hodnoty z neúplných alebo poškodených vstupov. Tento prístup umožňuje modelu naučiť sa rekonštruovať chýbajúce informácie, čo je zásadné pre jeho schopnosť adaptovať sa na situácie, kde nie sú všetky dáta dostupné.

LSTM vrstva pre korekciu poradia sa zameriava na obnovu správneho poradia zmiešaných segmentov. Táto schopnosť je kritická pre analýzu sekvencií, kde poradie dát môže byť narušené, čo je bežné v reálnych dátových sadách.

Výstupy z oboch LSTM vrstiev sú následne spracované plne prepojenými vrstvami, ktoré transformujú LSTM výstupy na predikcie. Tieto vrstvy umožňujú modelu vykonávať úsudky na vyššej úrovni a efektívne sa učiť komplexné mapovania medzi vstupnými funkciami a výstupmi. Tento integrálny prístup zaisťuje, že model je schopný nielen presne predpovedať a klasifikovať, ale aj efektívne rozpoznávať a reagovať na vzory a dynamiku vstupných sekvencií.

4.3.2 Tréningový proces

V rámci tréningového procesu svojho modelu som zvolil niekoľko kľúčových hyperparametrov a funkcií strát, ktoré boli navrhnuté tak, aby optimálne podporovali špecifické potreby a ciele mojej bakalárskej práce.

Hyperparametre

Počet vrstiev a rozmery skrytých jednotiek: Model obsahuje dve LSTM vrstvy s rozdielnymi dimenziami pre maskovacie a poradové úlohy - 200 jednotiek pre maskovaciu úlohu a 64 jednotiek pre úlohu korekcie poradia. Toto rozlíšenie umožňuje modelu efektívne spracovať a rozlišovať medzi dvoma typmi sekvenciálnych údajov, ktoré vyžadujú rôzne úrovne abstrakcie a detailu.

Funkcie strát

MSELoss (Mean Squared Error Loss) je použitý pre maskovaciu úlohu, kde je cieľom minimalizovať kvadratickú chybu medzi predpovedanými a skutočnými hodnotami maskovaných sekvencií. Táto strata je vhodná pre úlohy, kde chceme, aby model presne rekonštruoval pôvodné hodnoty.

Pre predikciu poradia v mojom modeli využívam funkciu straty Cross Entropy Loss. Táto metóda je štandardne používaná v klasifikačných úlohách, keďže efektívne meria, ako dobre model predpovedá skutočné kategórie alebo poradie z dát.

Tento model využíva kombináciu maskovacej straty a straty pre predikciu poradia.

4.4 Model 2 - Kombinácia maskovania a kontrastného učenia

4.4.1 Architektúra modelu

Druhý model je navrhnutý s využitím LSTM sietí na efektívne spracovanie maskovaných sekvencií dát. Tento model využíva bidirekcionálne LSTM vrstvy, ktoré zabezpečujú hlbokšie pochopenie kontextu v dátach tým, že spracúvajú informácie z oboch smerov sekvencie. Výstupy z LSTM sú následne transformované plne prepojenou vrstvou, ktorá predpovedá maskované hodnoty, a využíva sa pooling vrstva na redukciu dimenzionality dát, čo zjednodušuje extrakciu črt. Táto architektúra umožňuje modelu presne rekonštruovať a predpovedať dáta, čo je kľúčové pre úlohy, kde je potrebné obnoviť pôvodné hodnoty z čiastočne skrytých alebo maskovaných vstupov.

Na rozdiel od prvého modelu, ktorý kombinuje maskovacie a korekčné úlohy poradia, druhý model, je špecificky navrhnutý a optimalizovaný výhradne pre predikciu maskovania.

4.4.2 Tréningový proces

Model je trénovaný pomocou kombinácie pretextovej úlohy maskovania a kontrastného učenia. Tento prístup využíva tri hlavné typy sekvencií:

- Anchor: Toto je pôvodná sekvencia dát, ktorá slúži ako základ pre porovnávanie.
- Pozitívny pár: Pozitívny pár pre anchor je pôvodná sekvencia, kde je 25 percent dát zamaskovaných. Tento pár je stále považovaný za "podobný" ku kotviacej sekvencii, pretože obsahuje len čiastočné maskovanie.
- Negatívny pár: Ako negatívny pár sa používa náhodná sekvencia z rovnakého batchu, ktorá nie je pôvodnou sekvenciou, s uplatnením maskovania 50 percent dát. Táto sekvencia je považovaná za "nepodobnú" kotviacej sekvencii.

Tieto sekvencie sú následne poslané do modelu, kde sa pomocou funkcií na extrakciu reprezentácií získavajú ich reprezentácie. Tieto reprezentácie sú následne použité v kontrastnej stratovej funkcii, ktorá vyhodnocuje, ako dobre model rozlišuje medzi podobnými a nepodobnými párovými sekvenciami.

Celkovo model kombinuje stratu z maskovacej úlohy a kontrastnú stratu, čo mu umožňuje efektívne sa učiť z rozličných typov sekvencií a zvyšuje jeho schopnosť generalizácie a presnosti pri identifikácii a spracovaní sekvencií v rámci rôznych scenárov a úloh.

4.5 Model 3 - Kombinácia klasifikácie a kontrastného učenia

Tretí model v mojej bakalárskej práci, je hybridný model, ktorý kombinuje prístupy supervizovaného učenia a kontrastného učenia. Tento model je špecificky navrhnutý na zlepšenie klasifikačných schopností prostredníctvom efektívnej integrácie dvoch rôznych techník učenia.

4.5.1 Architektúra modelu

Model používa tradičné supervizované učenie, kde sú k dispozícii označené dáta, na učenie predikcie tried na základe vstupných príznakov. Tento prístup umožňuje modelu efektívne naučiť sa rozpoznávať vzory a vzťahy v dátach, ktoré priamo súvisia s klasifikačnými labelmi.

Okrem supervizovaného učenia model zahŕňa aj kontrastné učenie, ktoré sa snaží zlepšiť schopnosť modelu rozlišovať medzi rôznymi kategóriami. Pri tomto prístupe model vyhodnocuje pary dát – pozitívne páry sú tvorené sekvenčnými páry so zhodnými labelmi (napríklad dve rôzne sekvencie z rovnakej triedy), zatiaľ čo negatívne páry sú tvorené sekvenčnými páry s rozdielnymi labelmi (napríklad sekvenčné páry z rôznych tried). Tento prístup posilňuje model tým, že ho učí identifikovať jemné rozdiely a podobnosti medzi triedami. Kombinácia supervizovaného a kontrastného učenia v jednom modeli zvyšuje jeho schopnosť generalizácie a presnosti pri klasifikácii. Supervizované učenie poskytuje priame smernice pre klasifikáciu, zatiaľ čo kontrastné učenie zlepšuje rozlišovacie schopnosti modelu a zvyšuje jeho odolnosť voči pretrénovaniu. Tento hybridný prístup umožňuje modelu lepšie porozumieť a interpretovať komplexné dátové štruktúry, čo je obzvlášť užitočné v aplikáciách, kde je dôležité presné rozlíšenie medzi podobnými ale rozdielnymi kategóriami.

4.6 Tréningový proces

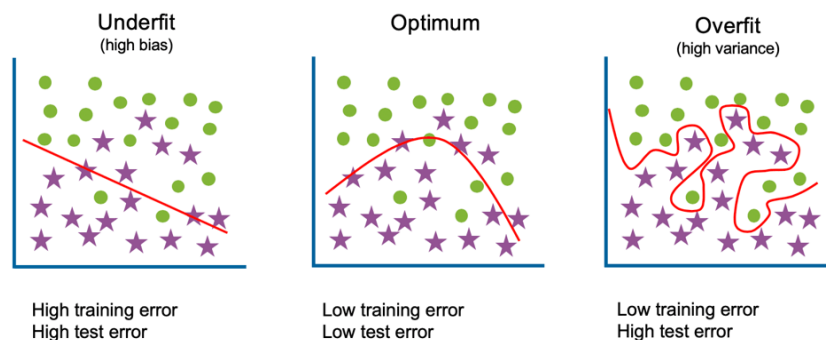
V tréningovom procese modelov v mojej bakalárskej práci sú implementované tri kľúčové metódy na optimalizáciu a efektívne učenie: Early Stopping, kombinácia tréningového a validačného datasetu, a Learning Rate Scheduler. Každá z týchto metód prináša špecifické výhody, ktoré zvyšujú efektívnosť a robustnosť modelu.

4.6.1 Early Stopping

Early Stopping je metóda používaná na predchádzanie pretrénovaniu modelu tým, že tréning je zastavený, keď sa validačná strata prestane zlepšovať po určitý počet epoch (definovaný ako patience). Táto technika monitoruje výkonnosť modelu na validačných dátach a zastaví tréning, ak sa nezaznamená žiadne zlepšenie. Early Stopping pomáha ušetriť čas a výpočtové zdroje tým, že zabráňuje zbytočne dlhému tréningu bez zlepšenia výkonnosti. Zároveň znižuje riziko pretrénovania, čo je dôležité pre schopnosť modelu generalizovať na nové, nevidené dáta.

4.6.2 Kombinácia tréningového a validačného datasetu

Modely sú trénované s použitím tréningového datasetu a ich výkonnosť je pravidelne overovaná na validačnom datasete. Tento prístup umožňuje sledovať, ako dobre model funguje na dátach, ktoré nebol počas tréningu videný, a poskytuje spätnú väzbu na to, ako optimalizovať model. Kombinácia tréningového a validačného datasetu zvyšuje schopnosť modelu správne fungovať v reálnych aplikáciách tým, že zabráňuje jeho prispôbaniu sa len na špecifické tréningové dáta. Poskytuje objektívnejší pohľad na výkonnosť modelu a umožňuje identifikovať a opraviť prípadné problémy s overfittingom.



Obr. 4.1: Příklad overfittingu pri trénovaní modelov [13]

4.6.3 Learning Rate Scheduler

Scheduler rýchlosti učenia automaticky upravuje learning rate (rýchlosť učenia) v priebehu tréningu. V tejto implementácii sa používa *ReduceLROnPlateau*, ktorý znižuje learning rate, keď validačná strata prestane klesať, čo indikuje, že model by mohol profitovať z pomalšieho a presnejšieho približovania sa k optimu.

Použitie scheduleru zvyšuje efektívnosť učenia tým, že prispôsobuje rýchlosť učenia na základe vývoja tréningu. Tento adaptívny prístup umožňuje modelu lepšie sa prispôbiť a efektívnejšie dosiahnuť optimálne riešenie, čo vedie k vyššej celkovej presnosti a výkonnosti modelu.

4.7 Extrakcia reprezentácií

V procese extrakcie reprezentácií, ktorý je aplikovaný vo všetkých modeloch v mojej práci, extrahujem reprezentácie pre všetky typy dátových sád: tréningovú, validačnú a testovaciu. Každá reprezentácia je získaná z modelu a je ukladaná do špecifických priečinkov pre jednotlivé typy dát. Tieto extrahované reprezentácie následne slúžia ako vstupy pre downstream úlohu. Proces extrakcie a ukladania reprezentácií vyzerá nasledovne:

- Extrakcia reprezentácií: Modely prechádzajú cez dáta poskytnuté cez *DataLoader*. Počas tohto procesu sa používa funkcia na extrakciu, ktorá transformuje dáta do nižšie-dimenzionálneho priestoru reprezentácií.
- Ukladanie reprezentácií: Po extrakcii sú reprezentácie uložené do diskových priečinkov, organizovaných podľa typu dát (tréningové, validačné, testovacie). Každá dávka reprezentácií je uložená ako samostatný súbor, čo umožňuje efektívne nahrávanie a spracovanie v neskorších fázach.
- Priradenie označení: K reprezentáciám sú priradené príslušné označenia (labels), ktoré indikujú webovú aplikáciu. Tieto označenia sú rovnako uložené a sú kľúčové pre supervizované učenie a evaluáciu modelov.
- Vlastný dataset a *DataLoader*: Reprezentácie a ich označenia sú načítané pomocou vlastne vytvorených tried *NetFlowDataset* a *DataLoader*, ktoré umožňujú iteratívne načítavanie a spracovanie dát počas tréningu alebo testovania modelov. Tento prístup zabezpečuje, že dáta sú spracované efektívne a v súlade s potrebami modelov.

4.8 Downstream úloha

V mojej bakalárskej práci používam downstream úlohu klasifikácie, ktorá sa zakladá na modeli ktorý je štruktúrovaný tak, aby zahrňoval enkodér na transformáciu vstupných vektorov príznakov do nižšie-dimenzionálneho priestoru a klasifikátor, ktorý vracia logity pre každú triedu. Klasifikátor je navrhnutý tak, aby poskytoval výstupy priamo vhodné pre výpočet straty pomocou `nn.CrossEntropyLoss`, ktorá zahrňa softmaxovú funkciu.

4.8.1 Tréningový proces

Pri tréningu modelu sa používa súbor dát získaný z extrahovaných reprezentácií, ktoré boli ukladané počas extrakčnej fázy z tréningových, validačných a testovacích datasetov. Načítanie týchto dát je zabezpečené vlastným datasetom a `DataLoaderom`. Tréning zahrňa použitie optimizéra Adam pre úpravu váh a `Learning Rate Scheduler` pre prispôbenie rýchlosti učenia v závislosti od pokroku v učení.

Optimizácia prebieha cez vypočítané straty, kde sa snaha o minimalizáciu straty cez epochy monitoruje a upravuje. Ak nedochádza k zlepšeniu validačnej straty, aktivuje sa mechanizmus `Early Stopping`, ktorý tréning predčasne ukončí a zamedzí tak pretrénovaniu.

4.8.2 Testovací proces

Po dokončení tréningu a validačných fáz, kde sa model snaží dosiahnuť čo najlepšie výsledky, nasleduje testovacia fáza. Tu model predikuje triedy pre testovací dataset a jeho výkon je vyhodnocovaný pomocou metrík ako presnosť (accuracy), precision, recall a F1 skóre. Tieto metriky poskytujú kvantitatívne hodnotenie účinnosti modelu pri klasifikácii.

4.8.3 Vyhodnocovanie výkonu

Výkon modelu sa vyhodnocuje taktiež na základe matice zámen (confusion matrix), ktorá ilustruje správnosť predikcií v porovnaní s aktuálnymi triedami. Analyzovaním tejto matice možno identifikovať, pre ktoré triedy model funguje dobre a kde má slabiny. Výsledky sú potom použité na úpravu a zlepšenie modelu.

Tento integratívny prístup zabezpečuje, že model je nielen schopný naučiť sa z dát, ale aj účinne ich klasifikovať a poskytovať presné predikcie, čo je základom pre jeho praktické využitie v reálnych aplikáciách.

Kapitola 5

Výsledky

5.1 Experiment.1 - Maskovacia strata vs premiešavacia strata

V mojej práci som realizoval ako prvý experiment, ktorý skúmal vplyv rôznych kombinácií váh strát pre pretextové úlohy maskovania a premiešavania na učenie modelu. Cieľom bolo identifikovať, ako rôzne proporcie týchto strát ovplyvňujú schopnosť modelu naučiť sa efektívne reprezentácie a následne ich využiť v downstream úlohách, konkrétne v klasifikácii.

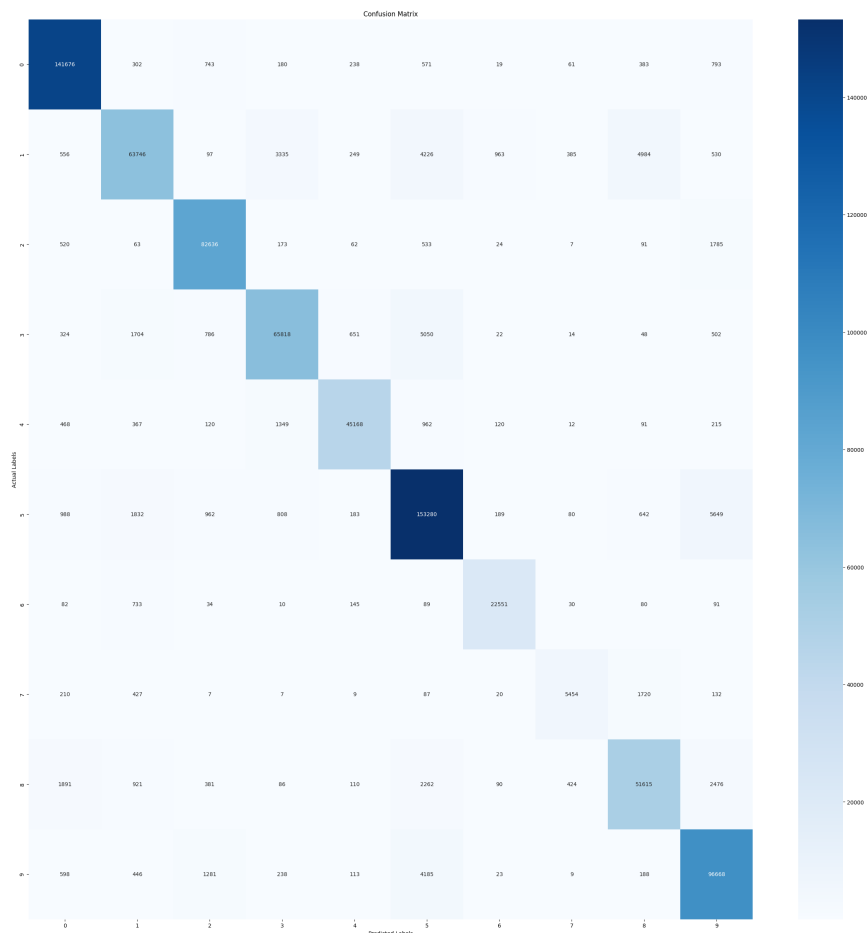
5.1.1 Experimentálny dizajn

Model bol trénovaný s použitím sériového prístupu, kde boli postupne menené váhy pre maskovaciú stratu a stratu premiešavania. Váhy týchto strát boli nastavené tak, aby ich súčet bol vždy rovný jednej, pričom sa menili v rozsahu od 0 do 1 s krokmi po 0.1. **Príklad:** V prvom behu experimentu bol pomer váh 0.1 pre maskovaciú stratu a 0.9 pre stratu premiešavania, nasledujúcom experimente to bolo 0.2 ku 0.8.

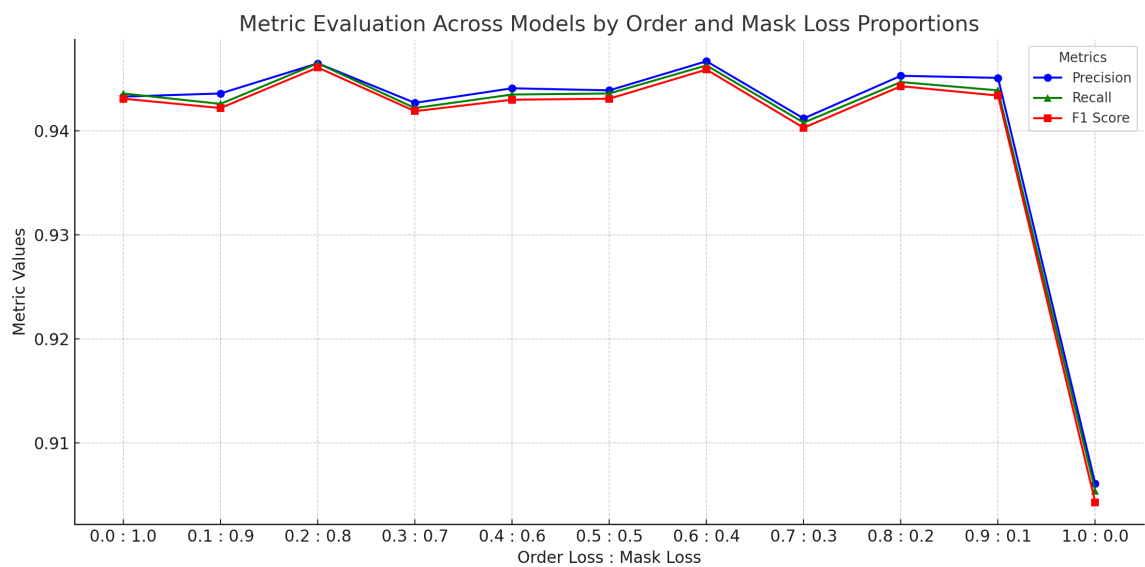
5.1.2 Hodnotenie výsledkov

Výsledky boli vyhodnotené na základe viacerých metrík, vrátane priemernej straty na testovacej sade, accuracy, precision, recall. Najlepšie výsledky boli zaznamenané pre model s pomerom váh 0.2 pre maskovaciú stratu a 0.8 pre stratu premiešavania. Konkrétne, tento model dosiahol nasledujúce výsledky:

- Váha maskovaciej straty: 0.2
- Váha premiešavaciej straty: 0.8
- Priemerná testovacia presnosť: 94.65%
- Precision: 0.9465
- Recall: 0.9465
- F1 skóre: 0.9461



Obr. 5.1: Matica zámien - Presnosť správnych predikcii aplikáci pre najvýkonnejší maskovacii a premiešavací model



Obr. 5.2: Graf vplyvu pomeru maskovaciej a premiešavaciej straty na výkon modelu

Tieto výsledky naznačujú, že správne nastavenie váh medzi rôznymi typmi strát môže výrazne ovplyvniť učenie a výkonnosť modelu. Zistenie, že nižšia váha maskovacej straty a vyššia váha premiešavacej straty vedie k lepším výsledkom, poskytuje cenné informácie pre návrh účinnejších tréningových schém. Tento experiment teda potvrdzuje hypotézu, že kombinácia rôznych strat môže zlepšiť schopnosť modelu generalizovať a efektívne sa učiť z komplexných dátových sád.

5.2 Experiment.2 - Maskovacia strata vs kontrastná strata

V druhom experimente mojej práce som skúmal vplyv kombinácie maskovacej a kontrastnej straty na výkon druhého modelu. Cieľom tohto experimentu bolo porovnať, ako rôzne váhové pomery týchto dvoch strat ovplyvňujú schopnosť modelu správne klasifikovať a rozlišovať dáta.

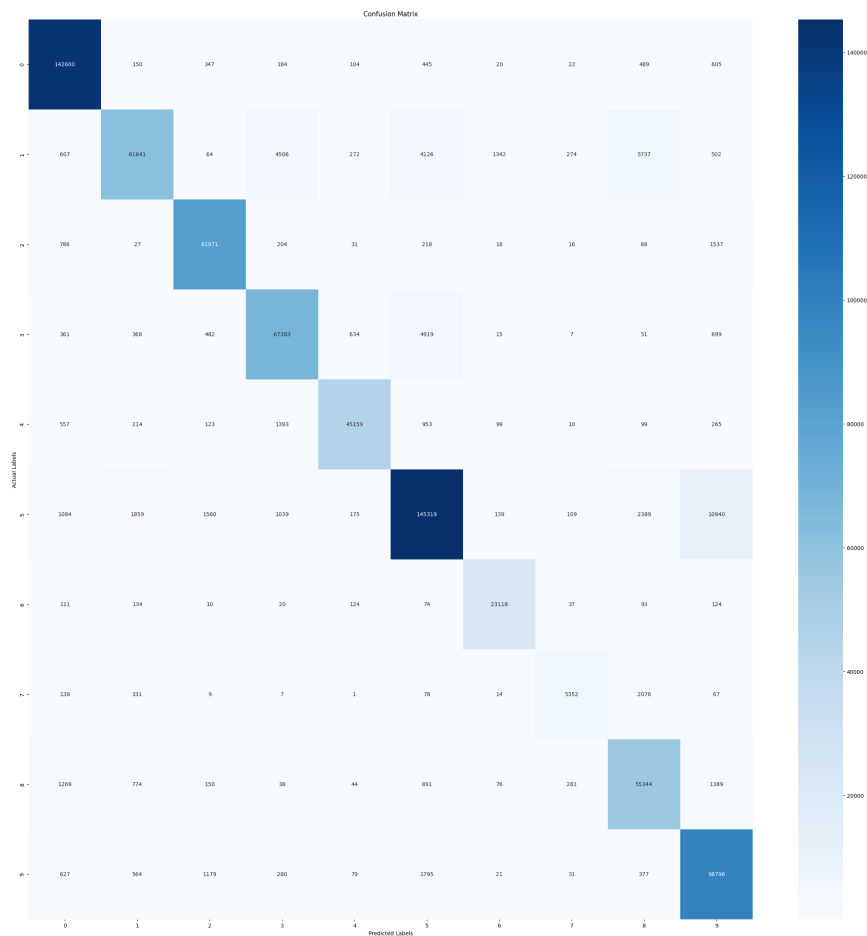
5.2.1 Experimentálny dizajn

Model bol trénovaný s rôznymi kombináciami váh pre maskováciu a kontrastnú stratu. Pomer váh sa menil podobne ako v prvom experimente, avšak tentokrát sa zvažoval vplyv pridania kontrastnej straty k maskovacej strate. Experimentálne nastavenia zahŕňali modely s rôznymi váhami od čisto maskovacej straty (1.0 pre maskovanie a 0.0 pre kontrastné) až po rôzne kombinácie, kde kontrastná strata mala väčší podiel.

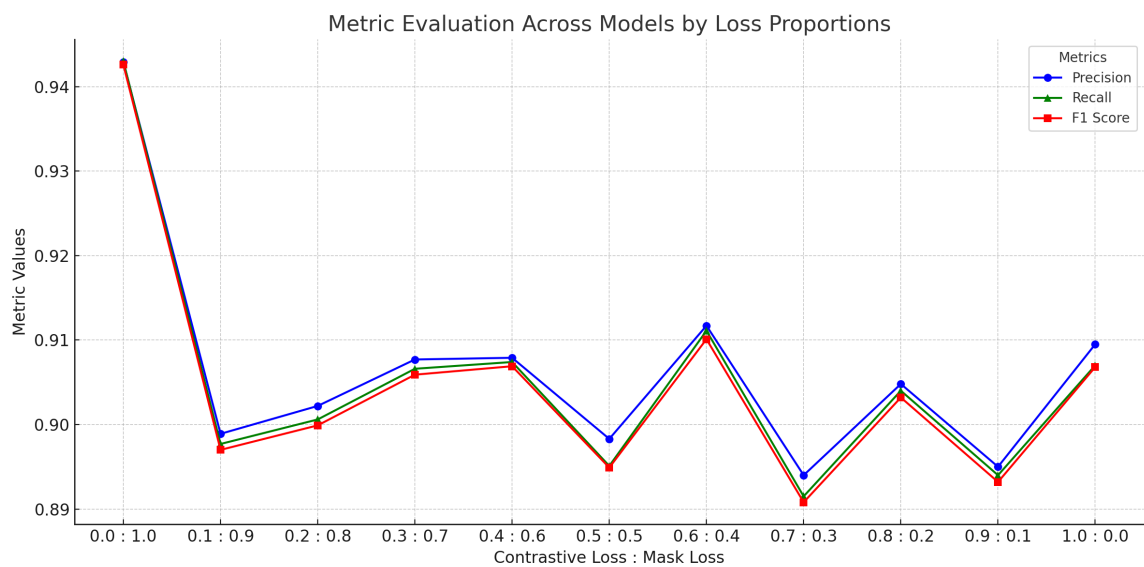
5.2.2 Hodnotenie výsledkov

Najlepšie výsledky bol zaznamenané pri modeli, ktorý používal výhradne maskováciu stratu, bez pridania kontrastnej straty:

- Váha maskovacej straty: 1
- Váha kontrastnej straty: 0
- Priemerná testovacia presnosť: 94.30%
- Precision: 0.9331
- Recall: 0.9325
- F1 skóre: 0.9319



Obr. 5.3: Matica zámien - Presnosť správnych predikcii aplikácii pre najvýkonnejší maskovacii a kontrastný model



Obr. 5.4: Graf vplyvu pomeru maskovaciej a kontrastnej straty na výkon modelu

Výsledky naznačujú, že pre daný dataset a úlohu mala maskovacia strata väčší prínos pre výkon modelu ako pridanie kontrastnej straty. To ukazuje, že v niektorých prípadoch môže byť jednoduchšie a cielenejšie zameranie na maskovaciu stratu účinnejšie než kombinovanie s kontrastnými prvkami. Tento poznatok je dôležitý pre ďalšie smerovanie vývoja a optimalizáciu modelov, kde by sa mala zväžiť efektivita jednotlivých stratových prístupov v kontexte špecifických úloh a dátových sád.

5.3 Experiment.3 - Klasifikačná strata vs kontrastná strata

V treťom experimente mojej bakalárskej práce som sa zamerlal na hodnotenie výkonu tretieho modelu, ktorý kombinuje straty z klasifikačnej úlohy a kontrastného učenia. Rovnako ako predtým, cieľom bolo zistiť, ako rôzne váhové pomery týchto strat ovplyvňujú schopnosť modelu správne klasifikovať dáta a rozlišovať medzi rôznymi kategóriami.

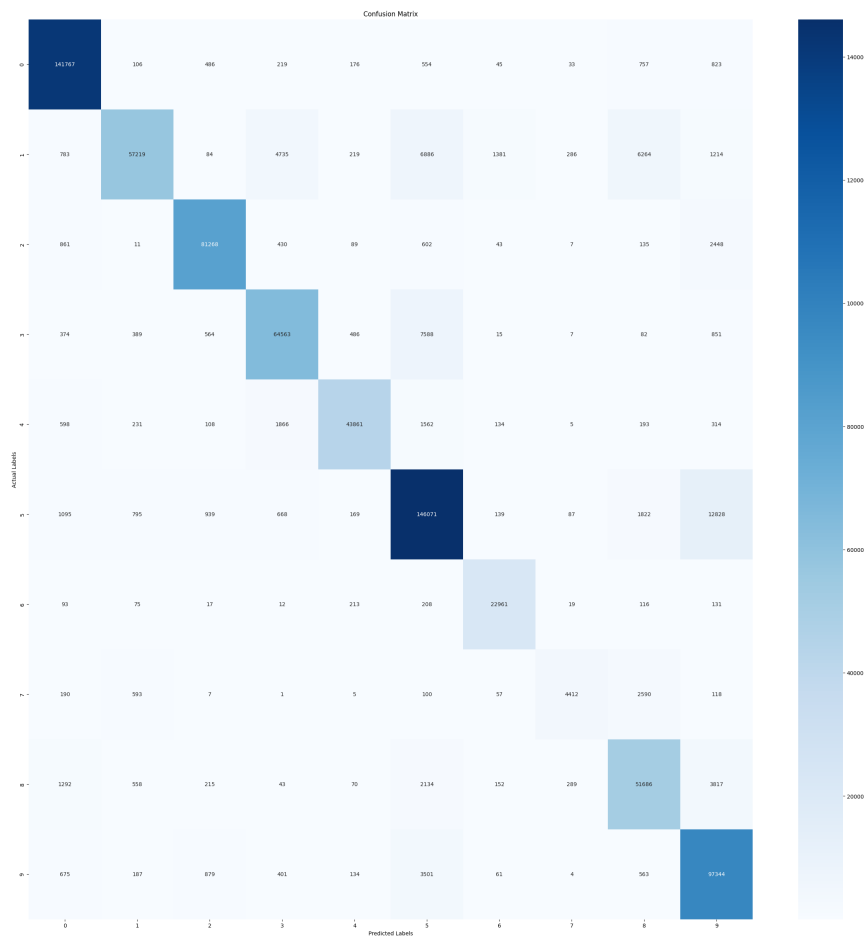
5.3.1 Experimentálny dizajn

Model bol trénovaný s rôznymi kombináciami váh pre klasifikačnú stratu a kontrastnú stratu. Pomer váh sa menil, pričom sa analyzovalo, ako zmena pomeru ovplyvňuje celkovú presnosť, stratu a ďalšie metriky výkonnosti modelu.

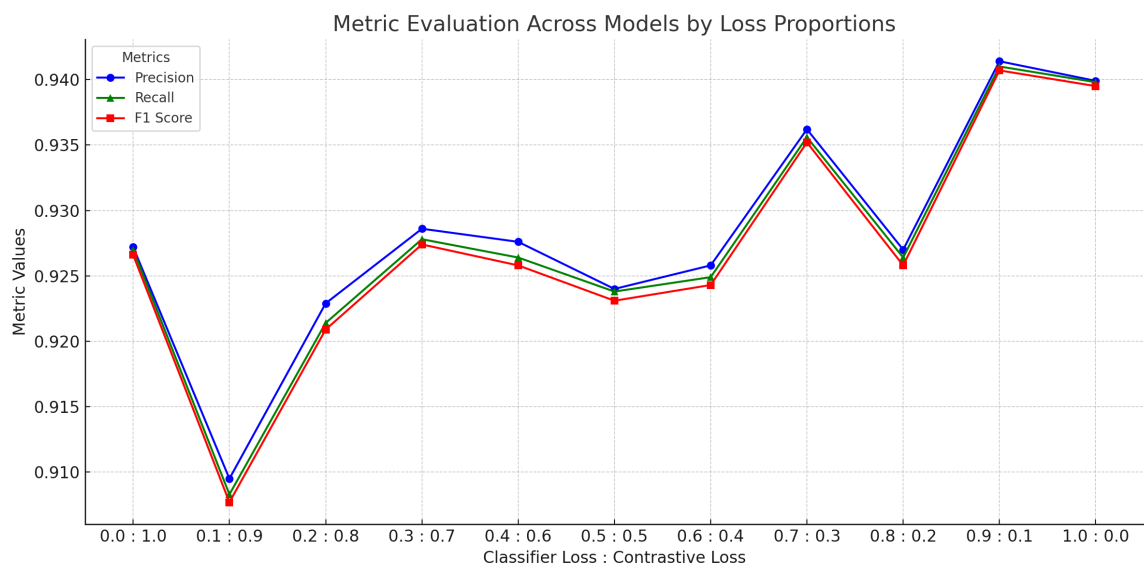
5.3.2 Hodnotenie výsledkov

Najlepšie výsledky bol dosiahnuté pre model s váhovým pomerom:

- Váha klasifikačnej straty: 0.9
- Váha kontrastnej straty: 0.1
- Priemerná presnosť na testovacej sade: 94.10%
- Presnosť (Precision): 0.9414
- Úplnosť (Recall): 0.9410
- F1 skóre: 0.9407



Obr. 5.5: Matica zámien - Presnosť správnych predikcií aplikáci pre najvýkonnejší klasifikačný a kontrastný model



Obr. 5.6: Graf vplyvu pomeru klasifikačnej a kontrastnej straty na výkon modelu

Tieto výsledky naznačujú, že dominantná klasifikačná strata sprevádzaná menšou mierou kontrastnej straty prináša najlepšiu rovnováhu pre model v kontexte zadaného experimentu. Vyššia váha pre klasifikačnú stratu zabezpečila, že model bol efektívne zameraný na klasifikáciu, zatiaľ čo prítomnosť kontrastnej straty v menšom pomere prispela k zlepšeniu rozlišovacej schopnosti modelu bez toho, aby to negatívne ovplyvnilo jeho hlavnú úlohu.

Kapitola 6

Záver

Táto bakalárska práca sa zamerala na reprezentáciu dát sieťových tokov z datasetov pomocou neurónových sietí, pričom využívala metódy samoučeného učenia a pretextové úlohy. Cieľom bolo porovnať vplyvy jednotlivých úloh a metód na výkon modelu pri downstreamovej úlohe klasifikácie webovej aplikácie. V rámci tejto práce sme sa zamerali na kombináciu váh stratových pre jednotlivé pretextové úlohy či metódy v rôznych modeloch, aby sa preskúmali ich jednotlivé a kombinované vplyvy.

Pri návrhu modelov sme najprv preštudovali teóriu týkajúcu sa self-supervised učenia, pretextových úloh a analýzy sieťových tokov. Základné koncepty strojového učenia, ako sú neurónové siete, aktivačné funkcie a optimalizačné algoritmy, nám poskytli pevný teoretický základ pre implementáciu. Potom sme sa zamerali na moderné metódy self-supervised učenia, ako sú kontrastné učenie, maskovanie dát a predikcia transformácií, aby sme preskúmali ich vplyv na kvalitu extrahovaných reprezentácií. Pretextové úlohy zahŕňali maskovanie a premiešavanie dát či identifikáciu chýbajúcich informácií, čo modelom pomohlo lepšie pochopiť vzťahy medzi rôznymi črtami sieťových tokov. Porovnávali sme teoretické poznatky s našimi výsledkami, čo nám pomohlo získať hlbší prehľad o efektívnosti kombinovaných metód. V návrhu implementácie tejto práce sa začalo navrhovaním troch modelov, ktoré kombinovali rôzne pretextové úlohy a metódy self-supervised učenia. Pretextové úlohy boli starostlivo vybrané a navrhnuté tak, aby umožnili modelom efektívne extrahovať reprezentácie z dát sieťových tokov. Modely boli implementované pomocou knižnice PyTorch, ktorá ponúka flexibilné a výkonné nástroje na prácu s neurónovými sieťami. Optimalizácia modelov sa uskutočnila s cieľom dosiahnuť čo najlepšie reprezentácie. V rámci experimentov a ich výsledkov boli tri navrhnuté modely testované na datasete CESNET TLS22 s cieľom preskúmať vplyv rôznych kombinácií stratových funkcií na výkonnosť modelov pri riešení downstream úloh. V prvom experimente, ktorý porovnával účinnosť maskovacej a premiešavacej straty, sa ukázalo, že maskovacia strata poskytuje modelu relevantnejšie reprezentácie z dát, čo viedlo k lepšej presnosti klasifikácie sieťových tokov. Výsledky boli podobné pre rôzne pomery stratových funkcií, a aj keď hlavný vplyv na výkonnosť mala maskovacia strata, model s najlepšími výsledkami kombinoval maskovaciu stratu s jemným pomerom premiešavacej straty.

Druhý experiment sa zamerl na kombináciu maskovacej a kontrastnej straty. Táto kombinácia umožnila modelu získať robustnejšie a komplexnejšie reprezentácie, čo viedlo k zlepšeniu výkonnosti pri identifikácii rôznych aplikačných značiek v sieťových tokoch. Výsledky však zároveň ukázali, že model dosiahol neporovnateľne najlepší výsledok pri využití výlučne maskovacej straty bez kontrastnej, keďže táto kombinácia nepriniesla očakávané zlepšenie a v niektorých prípadoch sa ukázala ako zbytočná.

Tretí experiment bol vykonaný na hybridnom modeli, kde sa kombinovalo supervised učenie s kontrastným učením s cieľom zistiť vplyv kombinácie kontrastného učenia na klasický klasifikátor s označeniami. Výsledky ukázali, že klasifikačná strata priniesla výrazné zvýšenie výkonnosti a efektivity modelu pri riešení špecifických klasifikačných úloh. Zároveň kontrastné učenie zabezpečilo robustnejšie reprezentácie, ktoré sú vhodné pre širšie spektrum bezpečnostných úloh v analýze sieťových tokov.

Celkovo experimenty preukázali, že rôzne kombinácie váh stratových funkcií majú výrazný vplyv na kvalitu získaných reprezentácií. Kombinácia jednotlivých metód a úloh môže v niektorých prípadoch výrazne zlepšiť výkonnosť modelov, zatiaľ čo v iných prípadoch sa ukázala ako zbytočná. Tieto zistenia podčiarkujú dôležitosť starostlivej optimalizácie váh a stratových funkcií pri kombinácii pretextových úloh a metód, aby sa dosiahol čo najlepší výkon na downstream úlohách.

Na základe týchto výsledkov práca odporúča pokračovať v skúmaní a rozširovaní týchto techník. Zahrnutie pokročilejších architektúr neurónových sietí a ďalších zdrojov dát môže pomôcť pri vývoji robustnejších modelov. Taktiež by bolo prínosné skúmať kombináciu prediktívnych úloh, zhľukovania a ďalších prístupov, aby sa zlepšila schopnosť modelov generalizovať v rôznych sieťových prostrediach a zvyšovala ochrana pred novými kybernetickými hrozbami. Týmto spôsobom môžu modely strojového učenia prispieť k lepšej identifikácii a ochrane pred komplexnými sieťovými útokmi v dynamickom prostredí kybernetickej bezpečnosti.

Literatúra

- [1] AI, M. *PyTorch* [<https://github.com/pytorch/pytorch>]. 2.1.0+cu121. 2024.
- [2] BALESTRIERO, R., IBRAHIM, M., SOBAL, V., MORCOS, A., SHEKHAR, S. et al. *A Cookbook of Self-Supervised Learning*. Technical Report. Meta AI, FAIR and New York University and University of Maryland and University of California, Davis and Université de Montreal, Mila and Univ Gustave Eiffel, CNRS, LIGM and Univ. Rennes, Inria, CNRS, IRISA, 2023.
- [3] BARTOŠ, V. a ZÍKA, J. *CESNET-TLS22: Dataset of Application-Layer Security Protocol Traffic from the CESNET Network (2022-2023)*. Zenodo, 2023. DOI: 10.5281/zenodo.7965515. Dostupné z: <https://zenodo.org/record/7965515>.
- [4] BISHOP, C. M. *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006. ISBN 978-0-387-31073-2.
- [5] CESNET. *CESNET DataZoo Library*. 2024. Dostupné z: <https://github.com/CESNET/datazoo>.
- [6] CHEN, T., KORNBLITH, S., NOROUZI, M. a HINTON, G. *A Simple Framework for Contrastive Learning of Visual Representations*. 2020.
- [7] GEEKSFORGEEKS. *Neural Networks - A Beginner's Guide*. 2023. Dostupné z: <https://www.geeksforgeeks.org/neural-networks-a-beginners-guide/>.
- [8] IBM. *Machine Learning*. 2023. Dostupné z: <https://www.ibm.com/topics/machine-learning>.
- [9] IBM. *Decision Trees*. 2024. Dostupné z: <https://www.ibm.com/topics/decision-trees>.
- [10] IBM. *Random Forest*. 2024. Dostupné z: <https://www.ibm.com/topics/random-forest>.
- [11] IBM. *Neural Networks*. Accessed: 2024. Dostupné z: <https://www.ibm.com/topics/neural-networks>.
- [12] IBM. *Random Forests*. Accessed: 2024. Dostupné z: <https://www.ibm.com/topics/random-forest>.
- [13] JAVATPOINT. *Overfitting in Machine Learning*. 2023. Dostupné z: <https://www.javatpoint.com/overfitting-in-machine-learning>.

- [14] JUPYTER PROJECT. *Project Jupyter: Computational Narratives as the Engine of Collaborative Data Science*. Dostupné z: <https://jupyter.org/>.
- [15] KINGMA, D. P. a BA, J. Adam: A Method for Stochastic Optimization. *ArXiv preprint arXiv:1412.6980*. 2014. Dostupné z: <https://arxiv.org/pdf/1412.6980.pdf>.
- [16] LUXEMBURK, J. a ČEJKA, T. Fine-grained TLS services classification with reject option. *Computer Networks*. 2023, zv. 220, s. 109467. DOI: <https://doi.org/10.1016/j.comnet.2022.109467>. ISSN 1389-1286. Dostupné z: <https://www.sciencedirect.com/science/article/pii/S1389128622005011>.
- [17] NEPTUNE.AI. *Self-Supervised Learning: What It Is, What It's Not, and How It Relates to Self-Play*. 2023. Dostupné z: <https://neptune.ai/blog/self-supervised-learning>.
- [18] NOROOZI, M., VINJIMOOR, A., FAVARO, P. a PIRSIYAVASH, H. Boosting self-supervised learning via knowledge transfer. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, s. 9359–9367. Dostupné z: https://openaccess.thecvf.com/content_cvpr_2018/papers/Noroozi_Boosting_Self-Supervised_Learning_CVPR_2018_paper.pdf.
- [19] PANDYA, K. *Loss Functions and Optimizers in ML Models*. 2021. Dostupné z: <https://medium.com/geekculture/loss-functions-and-optimizers-in-ml-models-b125871ff0dc>.
- [20] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011, zv. 12, s. 2825–2830.
- [21] R-BLOGGERS. *Building a Neural Net from Scratch Using R - Part 1*. 2020. Dostupné z: <https://www.r-bloggers.com/2020/07/building-a-neural-net-from-scratch-using-r-part-1/>.
- [22] RAMACHANDRAN, R. *Activation Functions in Neural Networks*. 2017. Dostupné z: <https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6>.
- [23] RAMESH, A., PAVLOV, M., GOH, G., GRAY, S., VOSS, C. et al. Zero-shot Text-to-Image Generation. *ArXiv preprint arXiv:2109.14545*. 2021. Dostupné z: <https://arxiv.org/abs/2109.14545>.
- [24] RUMELHART, D. E., HINTON, G. E. a WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*. Springer Nature. 1986, zv. 323, č. 6088, s. 533–536. DOI: 10.1038/323533a0.
- [25] SAHU, S. *Optimizers in Machine Learning*. 2021. Dostupné z: <https://medium.com/nerd-for-tech/optimizers-in-machine-learning-f1a9c549f8b4>.
- [26] STACK OVERFLOW USER. *Is there any difference between an activation function and a transfer function?* 2014. Dostupné z: <https://stackoverflow.com/questions/26058022/is-there-any-difference-between-an-activation-function-and-a-transfer-function/37879926>.

- [27] TEAM, G. L. *Types of Neural Networks - An Overview*. 2021. Dostupné z: <https://www.mygreatlearning.com/blog/types-of-neural-networks/>.
- [28] TOWARDS DATA SCIENCE. *A Gentle Introduction to Neural Networks Series — Part 1*. 2019. Dostupné z: <https://towardsdatascience.com/a-gentle-introduction-to-neural-networks-series-part-1-2b90b87795bc>.
- [29] UC BERKELEY SCHOOL OF INFORMATION. *What is Machine Learning?* 2022. Accessed: 2024-04-18. Dostupné z: <https://ischoolonline.berkeley.edu/blog/what-is-machine-learning/>.
- [30] V7 LABS. *Contrastive Learning Guide: Definition, Frameworks, and Methods*. 2023. Dostupné z: <https://www.v7labs.com/blog/contrastive-learning-guide>.
- [31] VANČO, T. *Self-supervised učení v aplikacích počítačového vidění*. 2021. 168 s. Diplomová práce. Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav automatizace a měřicí techniky. Vedúci práce: Ing. Ilona Janáko.
- [32] WIKIPEDIA CONTRIBUTORS. *Linear regression — Wikipedia, The Free Encyclopedia*. 2024. Dostupné z: https://en.wikipedia.org/w/index.php?title=Linear_regression&oldid=1222778350.
- [33] WIKIPEDIA CONTRIBUTORS. *Logistic regression — Wikipedia, The Free Encyclopedia*. 2024. Dostupné z: https://en.wikipedia.org/w/index.php?title=Logistic_regression&oldid=1221099910.
- [34] WIKIPEDIA CONTRIBUTORS. *Support vector machine — Wikipedia, The Free Encyclopedia*. 2024. Dostupné z: https://en.wikipedia.org/w/index.php?title=Support_vector_machine&oldid=1222546847.

Príloha A

Obsah elektronickej prílohy

Na priloženom CD sa nachádza bakalárska práca v pdf formáte. CD taktiež obsahuje jednotlivé kódy ku aplikáciám Self-supervised metód napísané v prostredí Jupyter Notebook a s koncovkou .ipynb.

```
xpyczl00.zip/
├── doc/
│   ├── xpyczl00_representation_of_network_flows_using_neural_networks
│   └── Zdrojové súbory pre vytvorenie technickej správy
└── src/
    ├── MaskingOrderModel.ipynb . Skript obsahujúci model kombinujúci maskovanie a
    │   premiešavanie
    ├── MaskingContrastiveModel.ipynb.....Skript obsahujúci model kombinujúci
    │   maskovanie a kontrastné učenie
    └── ClassifierContrastiveModel.ipynb .... Skript obsahujúci model kombinujúci
        klasifikáciu a kontrastné učenie
```